

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스 소개

DAY1	DAY2	DAY3	DAY4	DAY5
데이터 분석을 위한 기초 수학			데이터 집계 및 시각화	
DAY6	DAY7	DAY8	DAY9	DAY10
	미니 프로젝트	확률분포, AB테스트		
DAY11	DAY12	DAY13	DAY14	DAY15
머신러닝				
DAY16	DAY17	DAY18	DAY19	DAY20
딥러닝		파이널 프로젝트		

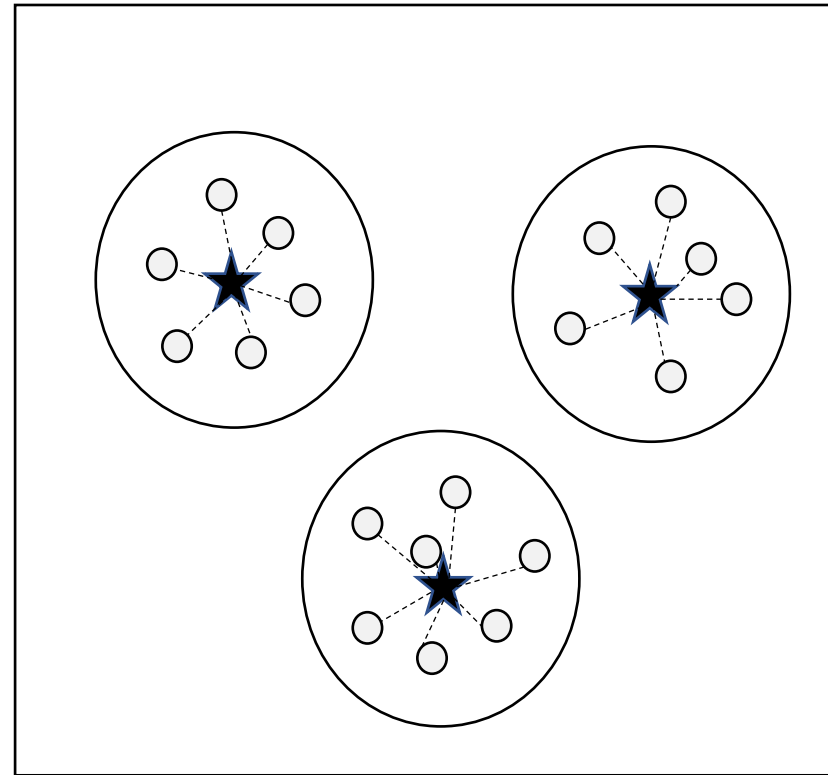
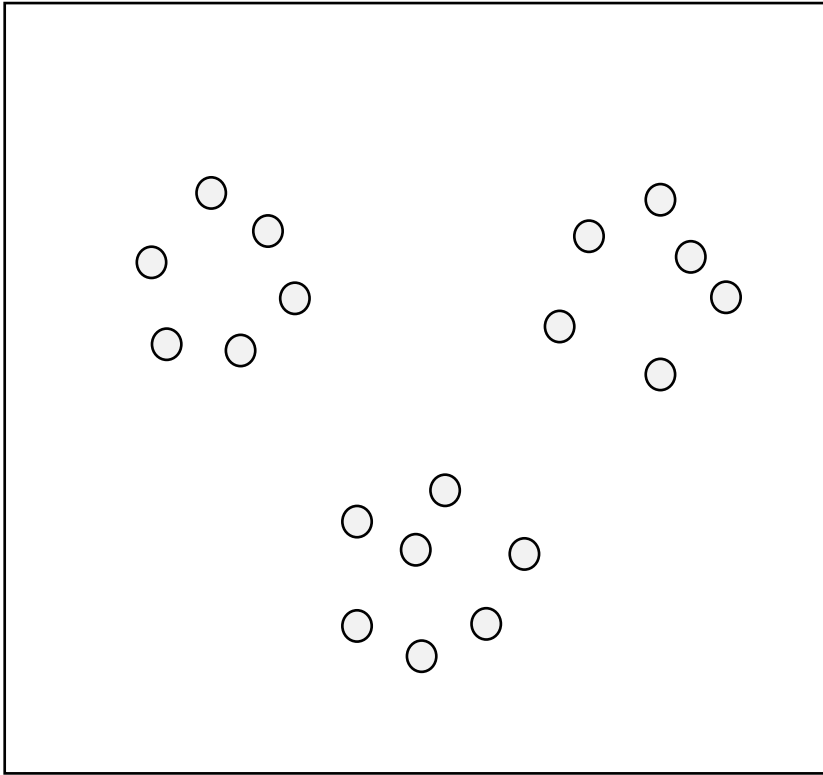
- ❑ k-means
- ❑ DBSCAN
- ❑ 가우시안 혼합 모형
- ❑ One Class SVM
- ❑ Local Outlier factor
- ❑ isolation forest
- ❑ 차원축소
- ❑ 주성분 분석
- ❑ LDA

AI/BigData 인텐시브 과정

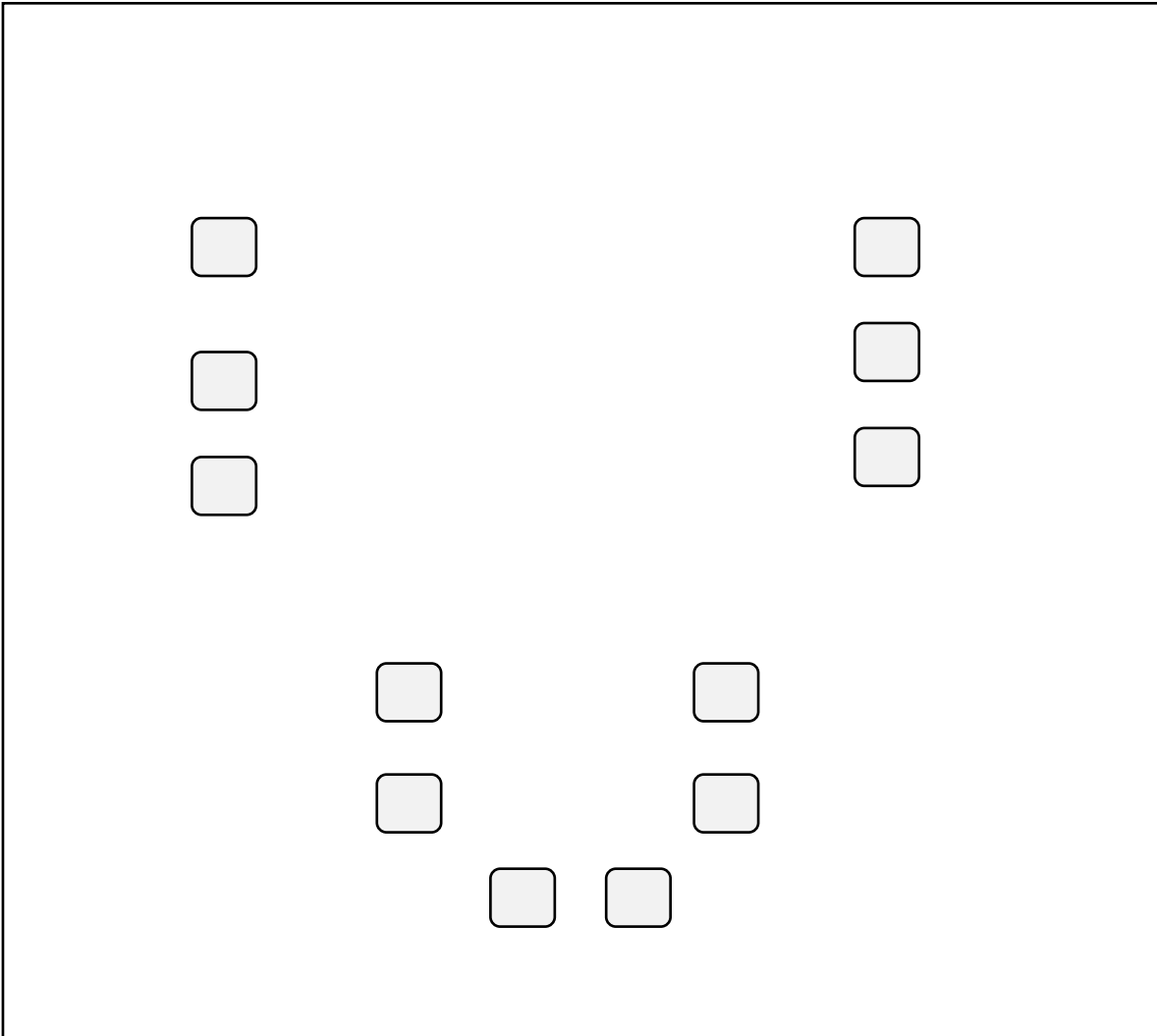
Section 1. k 평균 클러스터링

Section 1-1. k 평균 클러스터링 개념

k 평균 클러스터링(k means clustering)

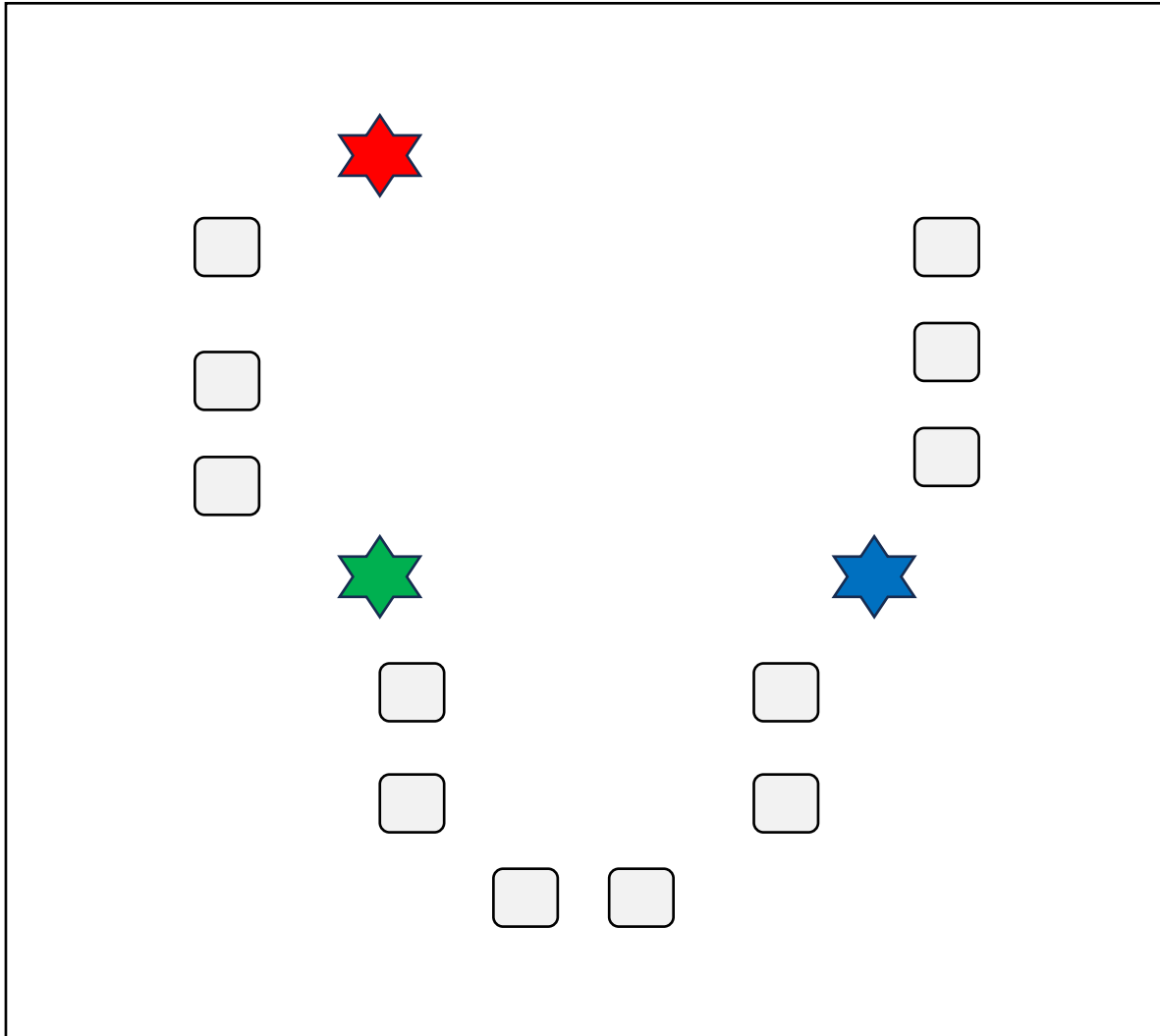


k 평균 클러스터링(k means clustering)



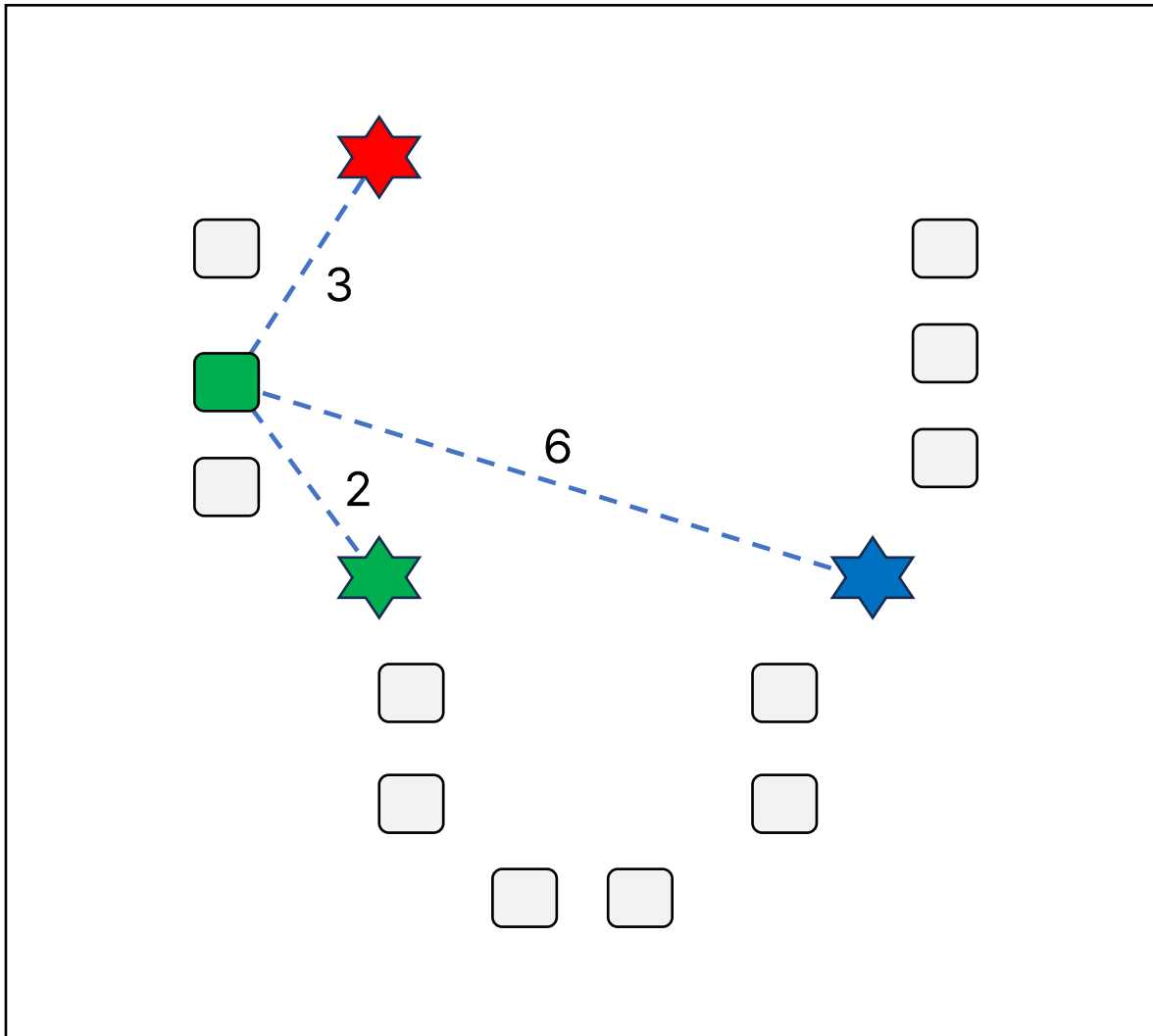
1. 사용자는 클러스터의 개수를 정함
2. 클러스터링할 데이터가 주어짐

k 평균 클러스터링(k means clustering)



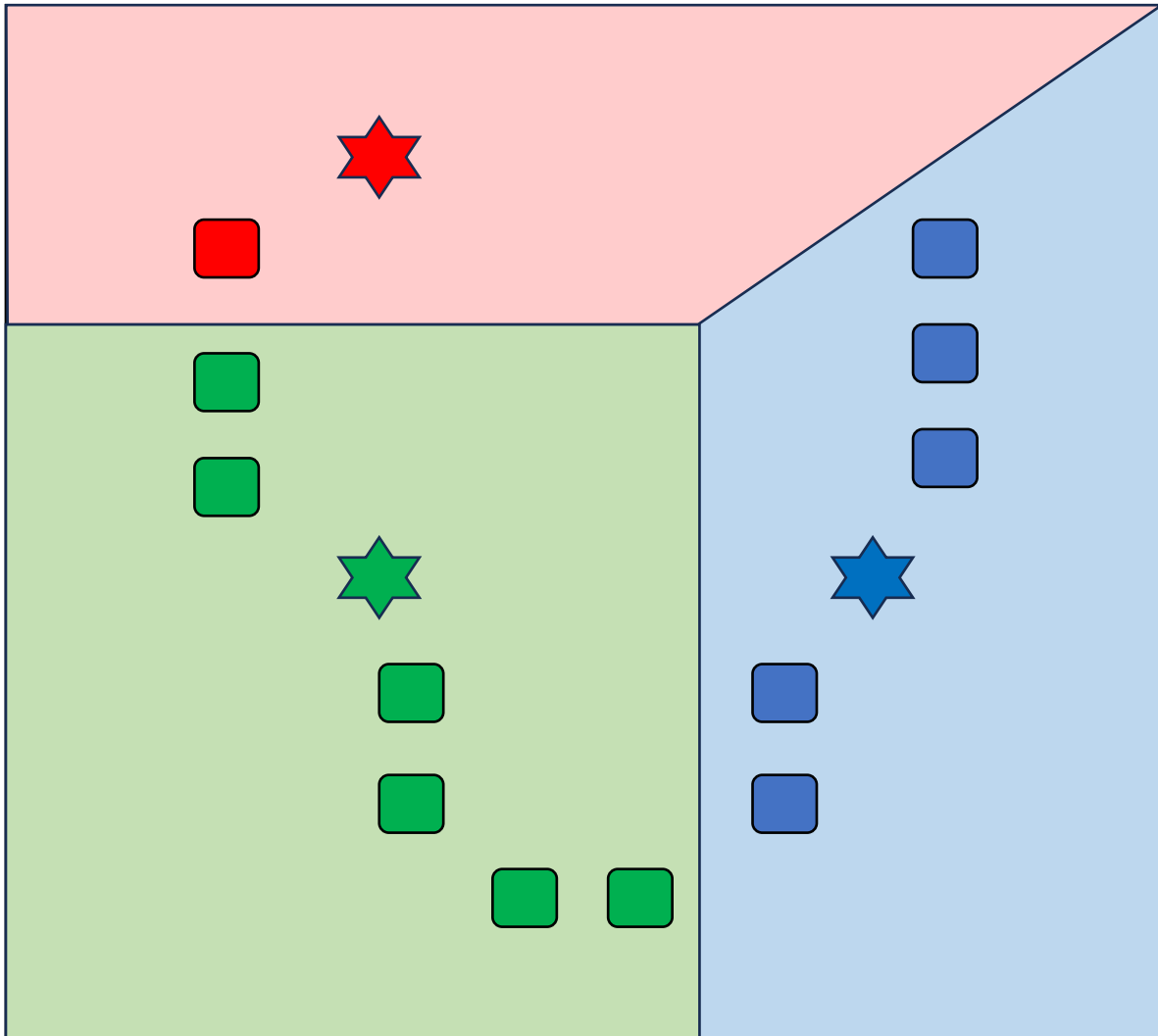
1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정

k 평균 클러스터링(k means clustering)



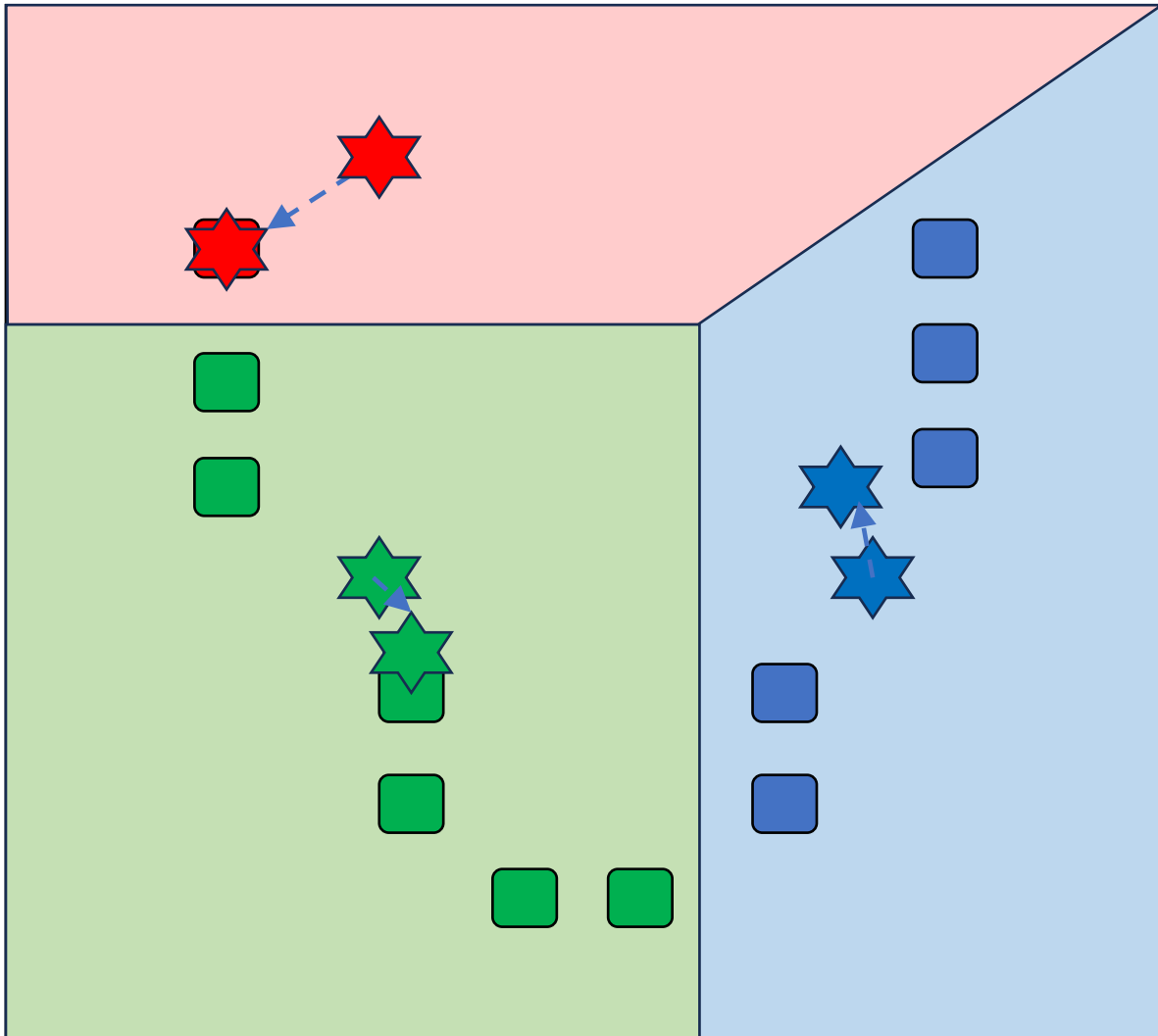
1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정
4. 각 데이터 포인트와 중심점과의 거리를 구한 후 가까운 곳으로 클러스터링

k 평균 클러스터링(k means clustering)



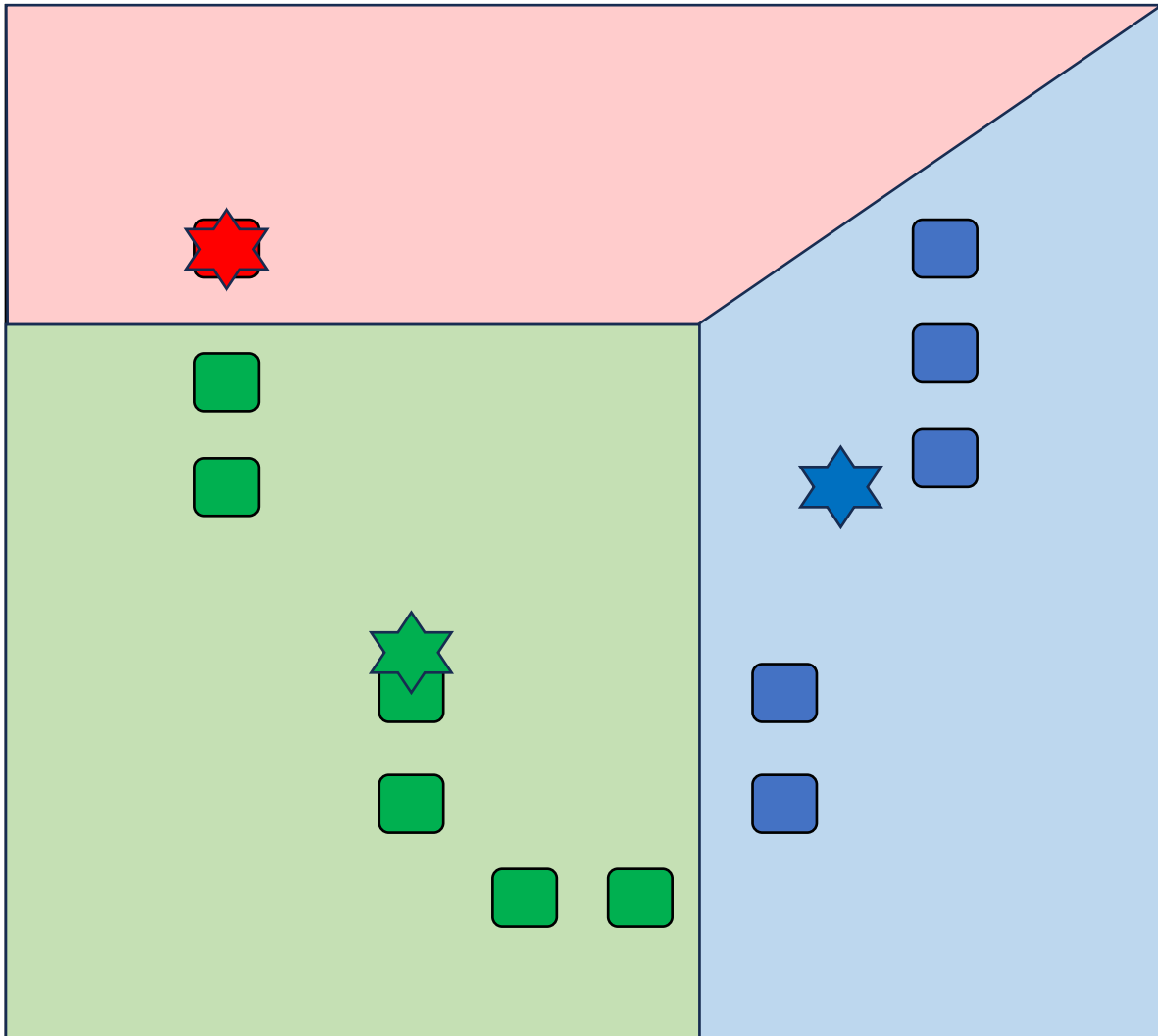
1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정
4. 각 데이터 포인트와 중심점과의 거리를 구한 후 가까운 곳으로 클러스터링

k 평균 클러스터링(k means clustering)



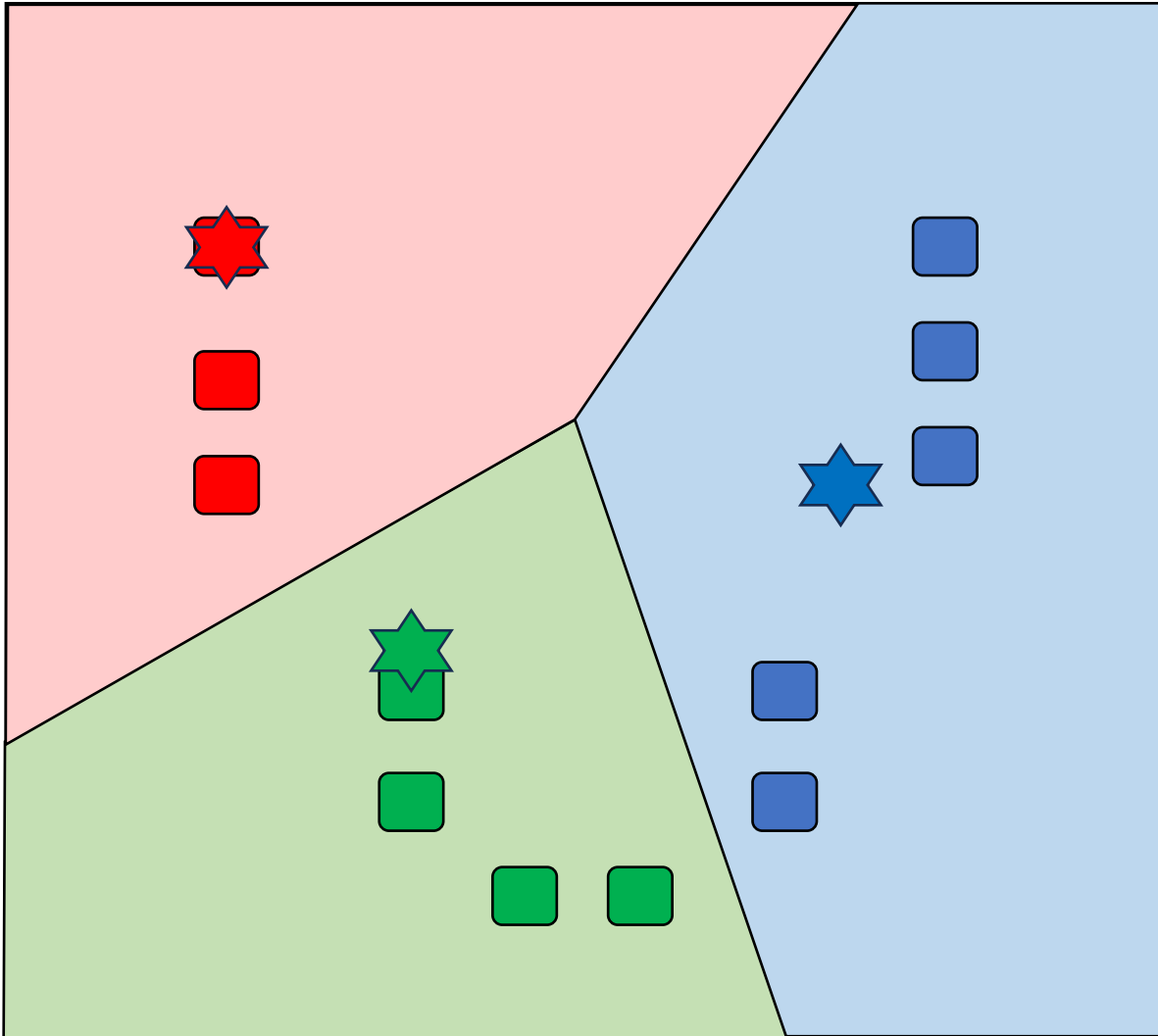
1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정
4. 각 데이터 포인트와 중심점과의 거리를 구한 후 가까운 곳으로 클러스터링
5. 중심점을 평균값으로 재조정

k 평균 클러스터링(k means clustering)



1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정
4. 각 데이터 포인트와 중심점과의 거리를 구한 후 가까운 곳으로 클러스터링
5. 중심점을 평균값으로 재조정

k 평균 클러스터링(k means clustering)



1. 사용자는 클러스터의 개수를 정함(이번 예제에서는 3으로 설정)
2. 클러스터링할 데이터가 주어짐
3. 랜덤으로 초기값 설정
4. 각 데이터 포인트와 중심점과의 거리를 구한 후 가까운 곳으로 클러스터링
5. 중심점을 평균값으로 재조정
6. 4로 돌아감

AI/BigData 인텐시브 과정

Section 1. k 평균 클러스터링

Section 1-2. k 평균 클러스터링 실습

k 평균 클러스터링(k means clustering)

In [1]: ▶

```
1 # 데이터 생성
2 from sklearn.datasets import make_blobs
3 X,y = make_blobs(n_samples=100,
4                  n_features=2,
5                  centers=5,
6                  random_state=10)
```

In [2]: ▶

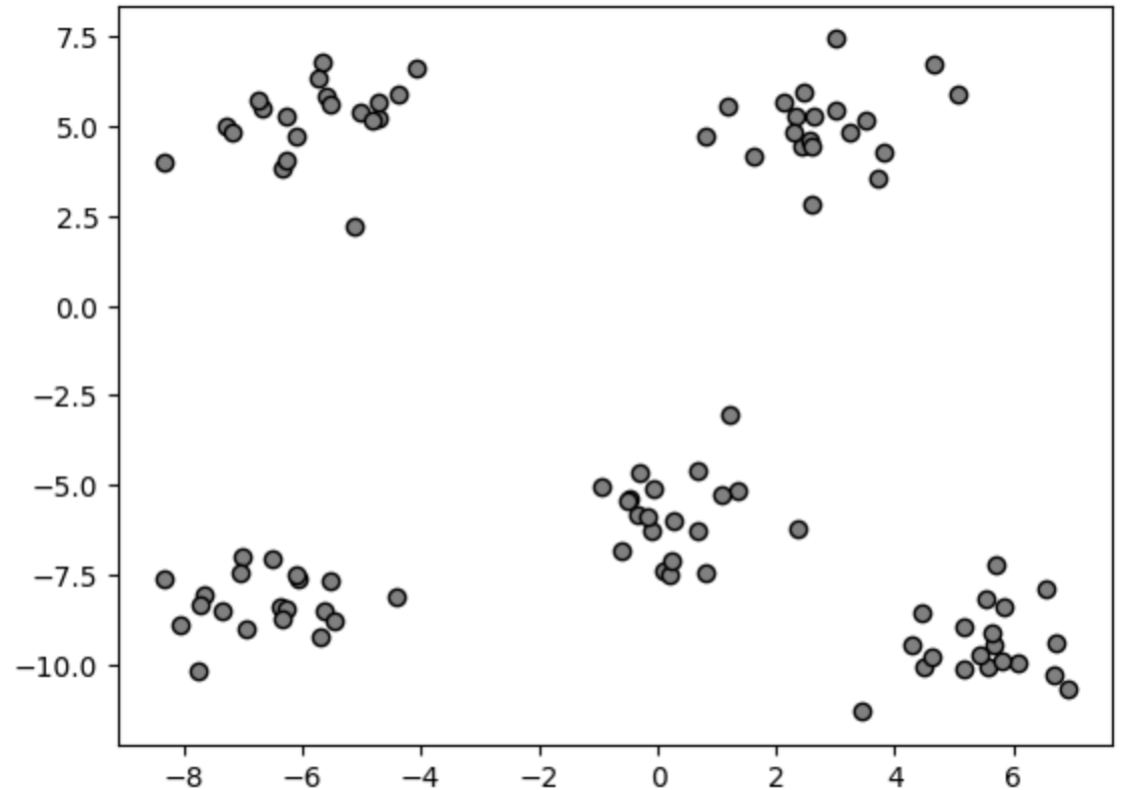
```
1 # 생성된 데이터 차원 확인
2 print(X.shape)
3 print(y.shape)
```

(100, 2)

(100,)

k 평균 클러스터링(k means clustering)

```
1 # 라벨링 되지 않은 데이터 플랏
2 import matplotlib.pyplot as plt
3 plt.scatter(X[:,0], X[:,1],
4             c='gray',
5             edgecolor='black',
6             marker='o')
7 plt.show()
```



k 평균 클러스터링(k means clustering)

In [13]: ▶

```
1 # k-means clustering
2 from sklearn.cluster import KMeans
3 kmc = KMeans(n_clusters=5,
4             init='random',
5             n_init=10,
6             max_iter=100,
7             random_state=0)
8 kmc.fit(X)
9 label_kmc = kmc.labels_
10 print(label_kmc)
```

```
[2 4 3 1 0 2 2 3 0 2 3 2 0 1 0 2 4 1 0 2 2 1 4 3 2 2 1 0 3 4 1 1 0 1 2 0 3
 1 3 4 1 1 1 0 3 3 2 0 4 0 4 0 2 3 0 0 1 4 1 3 0 4 4 1 2 4 0 4 0 3 3 2 4 0
 1 1 3 4 2 2 0 0 4 1 4 3 1 3 1 3 4 3 2 4 4 4 3 2 2 3]
```


k 평균 클러스터링(k means clustering)

In [14]: ▶

```
1 # 시각화를 위한 데이터프레임 생성
2 import pandas as pd
3 kmc_columns = ['kmc_comp1', 'kmc_comp2']
4 X_kmc_df = pd.DataFrame(X, columns=kmc_columns)
5 X_kmc_df['target'] = y
6 X_kmc_df['label_kmc'] = label_kmc
7 X_kmc_df.head(5)
```

Out[14]:

	kmc_comp1	kmc_comp2	target	label_kmc
0	-5.577854	5.872988	3	2
1	1.627832	4.178069	1	4
2	-6.950884	-9.024594	4	3
3	-0.952769	-5.034316	2	1
4	6.550104	-7.912339	0	0

In [15]: ▶

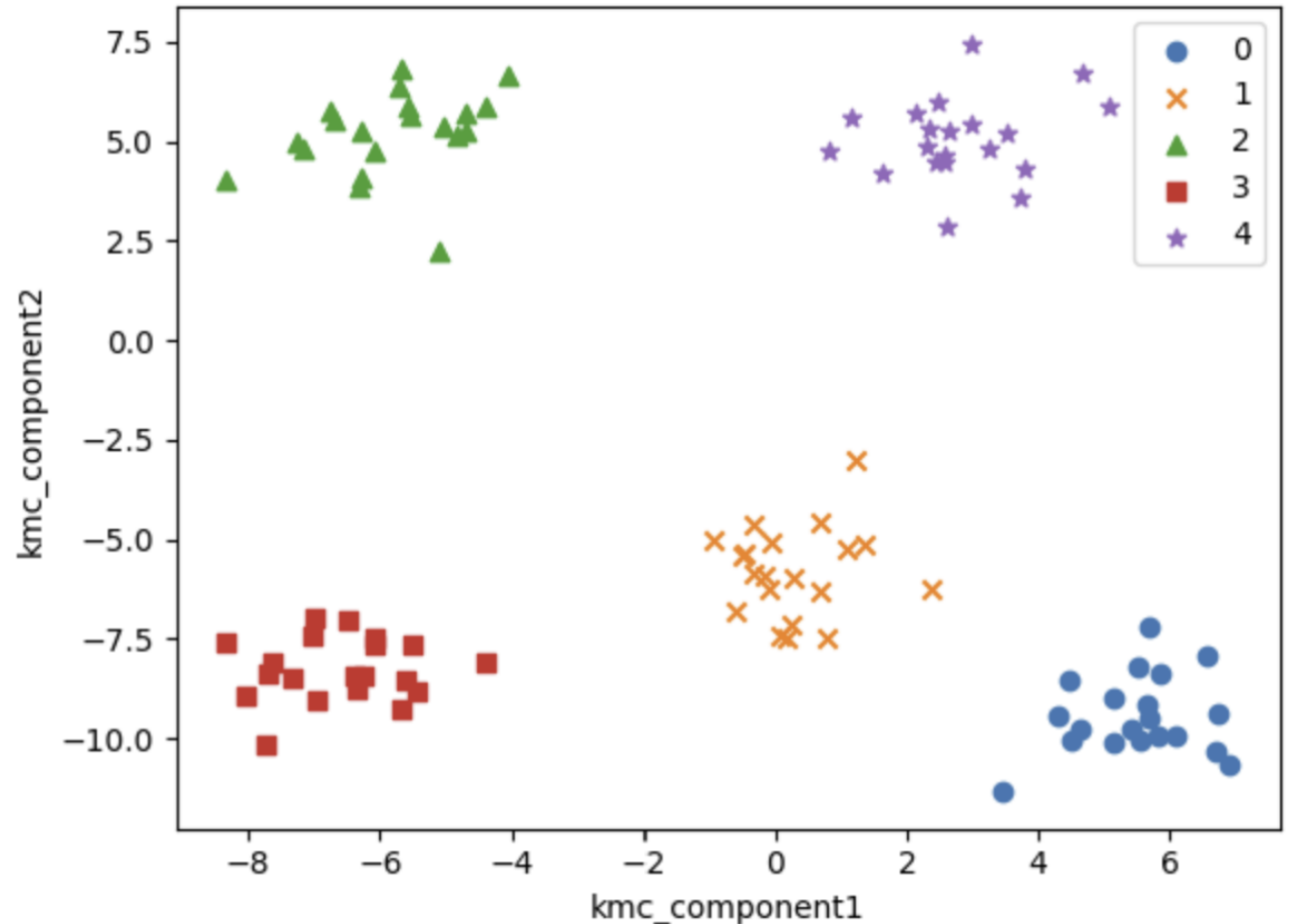
```
1 # 타겟 클래스 종류 확인
2 print(set(X_kmc_df['target']))
3 print(set(X_kmc_df['label_kmc']))
```

{0, 1, 2, 3, 4}

{0, 1, 2, 3, 4}

k 평균 클러스터링(k means clustering)

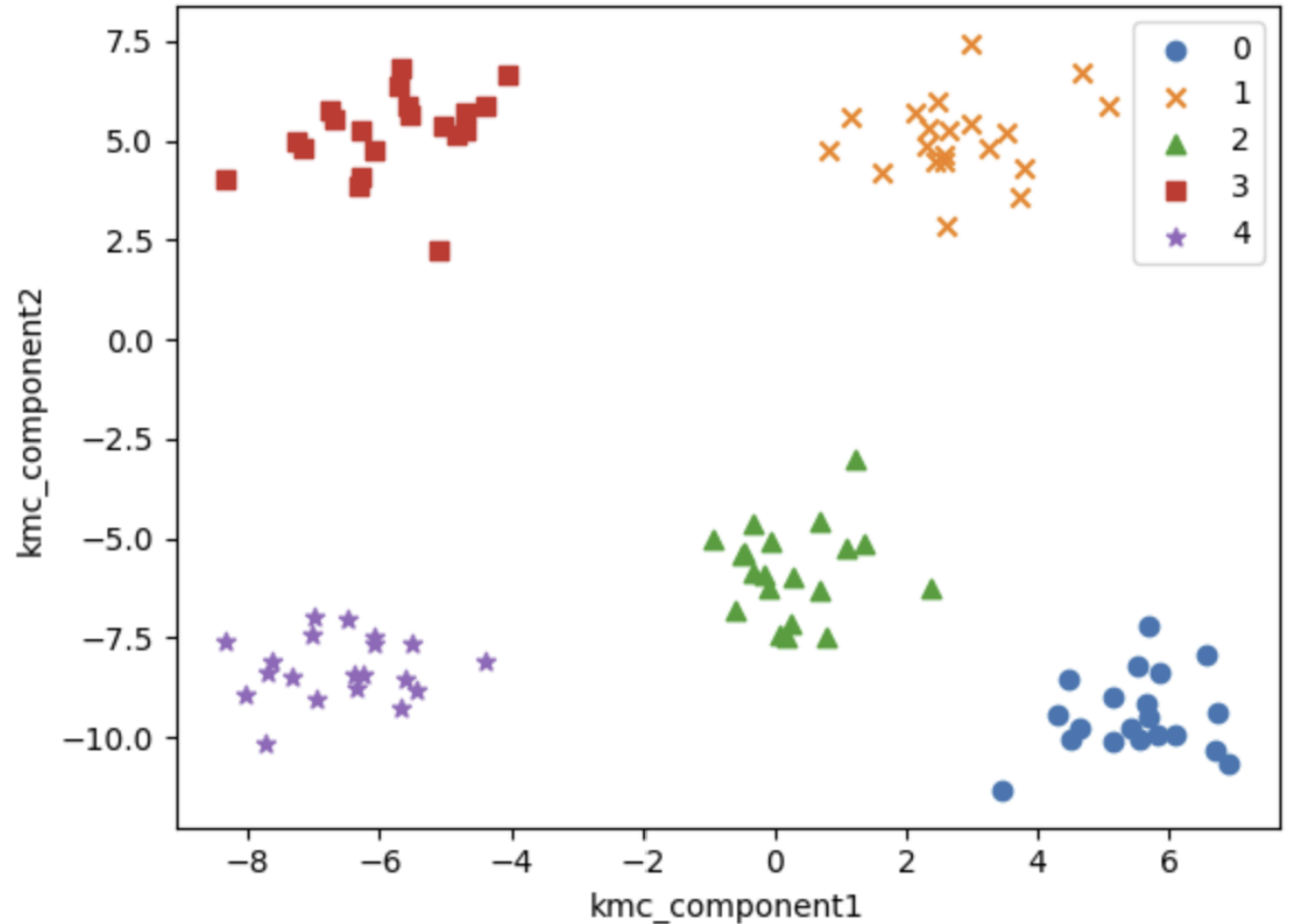
```
In [16]: ▶ 1 # k-means clustering을 이용한 데이터 플랏
2 df = X_kmc_df
3 markers=['o', 'x', '^', 's', '*']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['label_kmc']==i]
7     target_i = i
8     X1 = df_i['kmc_comp1']
9     X2 = df_i['kmc_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('kmc_component1')
15 plt.ylabel('kmc_component2')
16 plt.legend()
17 plt.show()
```



k 평균 클러스터링(k means clustering)

In [17]: ▶

```
1 # 실제 타겟 플랏
2 df = X_kmc_df
3 markers=['o', 'x', '^', 's', '*']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['target']==i]
7     target_i = i
8     X1 = df_i['kmc_comp1']
9     X2 = df_i['kmc_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('kmc_component1')
15 plt.ylabel('kmc_component2')
16 plt.legend()
17 plt.show()
```



k 평균 클러스터링(k means clustering)

In [18]: ▶

```
1 # 모형 평가
2 from sklearn.metrics import silhouette_score
3 sil_score = silhouette_score(X, label_kmc)
4 print(sil_score)
```

0.7598181300128782

실루엣 스코어(silhouette score)

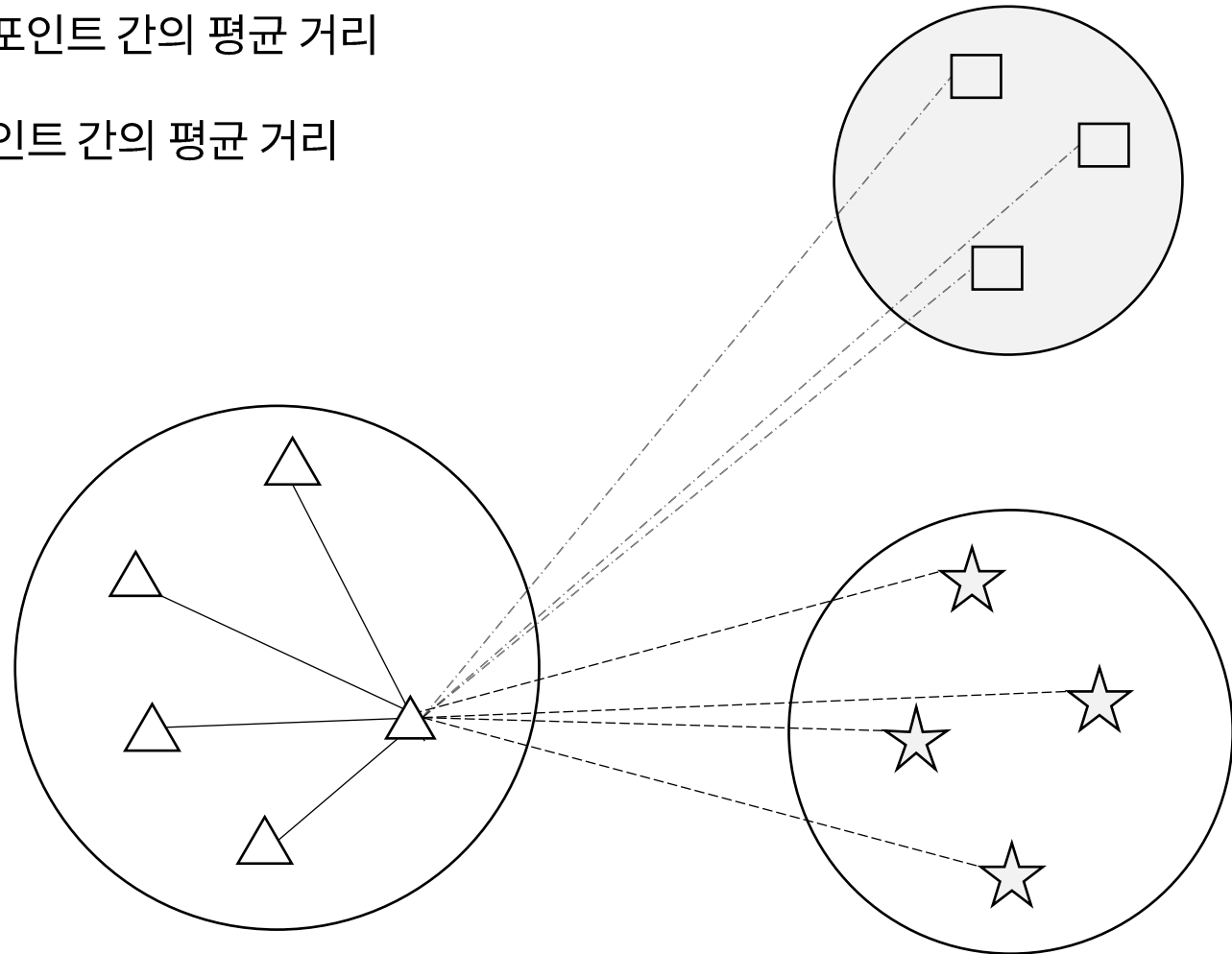
a_i i 번째 데이터 포인트와 동일한 클러스터에 속한 다른 데이터 포인트 간의 평균 거리

b_i i 번째 데이터 포인트와 다른 클러스터에 속한 다른 데이터 포인트 간의 평균 거리

$$silhouette_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

$$silhouette = \frac{1}{n} \sum_{i=1}^n silhouette_i$$

$$-1 \leq silhouette \leq 1$$

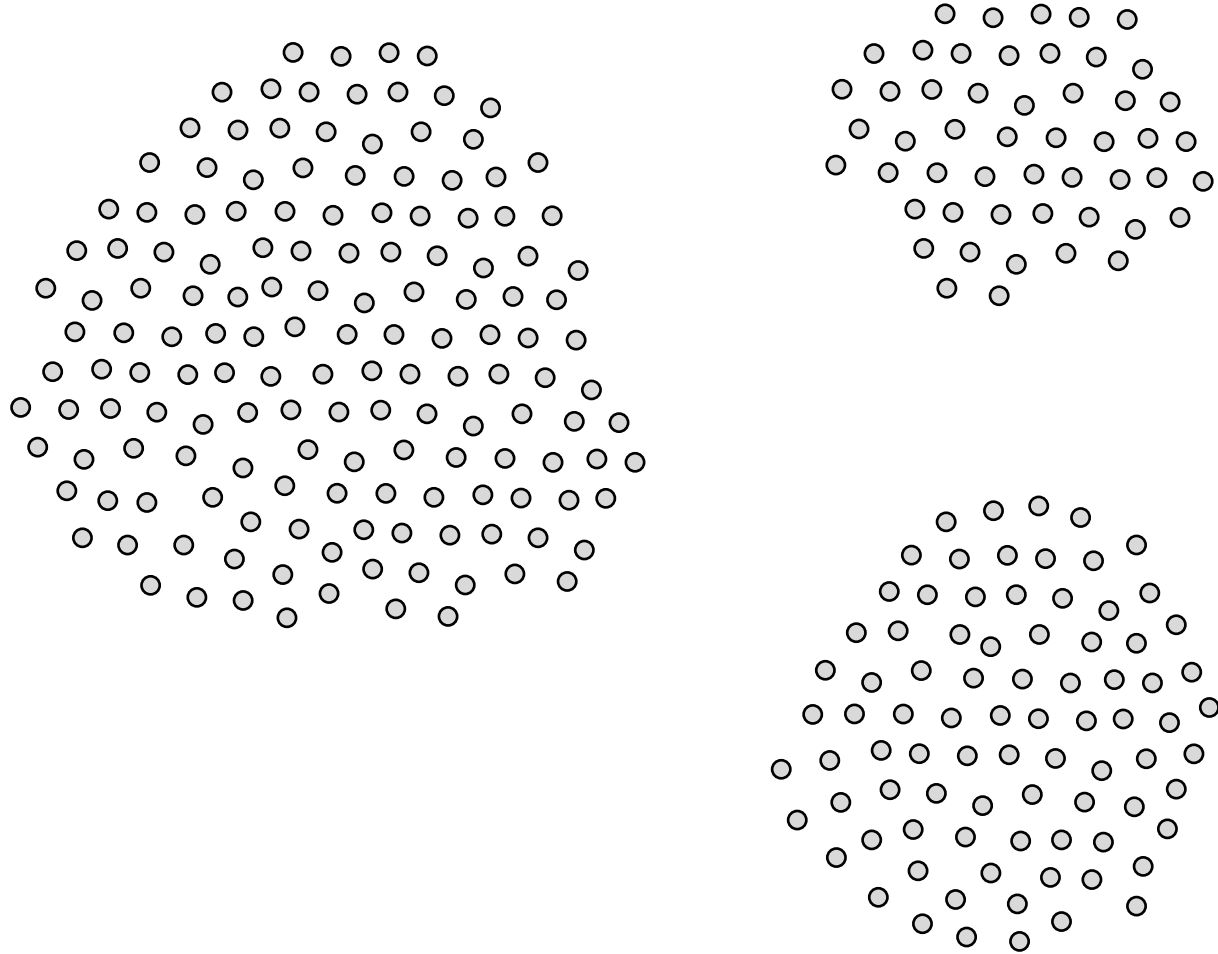


AI/BigData 인텐시브 과정

Section 2. DBSCAN

Section 2-1. DBSCAN 개념

DBSCAN 개념



AI/BigData 인텐시브 과정

Section 2. DBSCAN

Section 2-2. DBSCAN 실습

DBSCAN

In [1]: ▶

```
1 # 데이터 생성
2 from sklearn.datasets import make_moons
3 X,y = make_moons(n_samples=300,
4                   noise=0.05 ,
5                   random_state=0)
```

In [2]: ▶

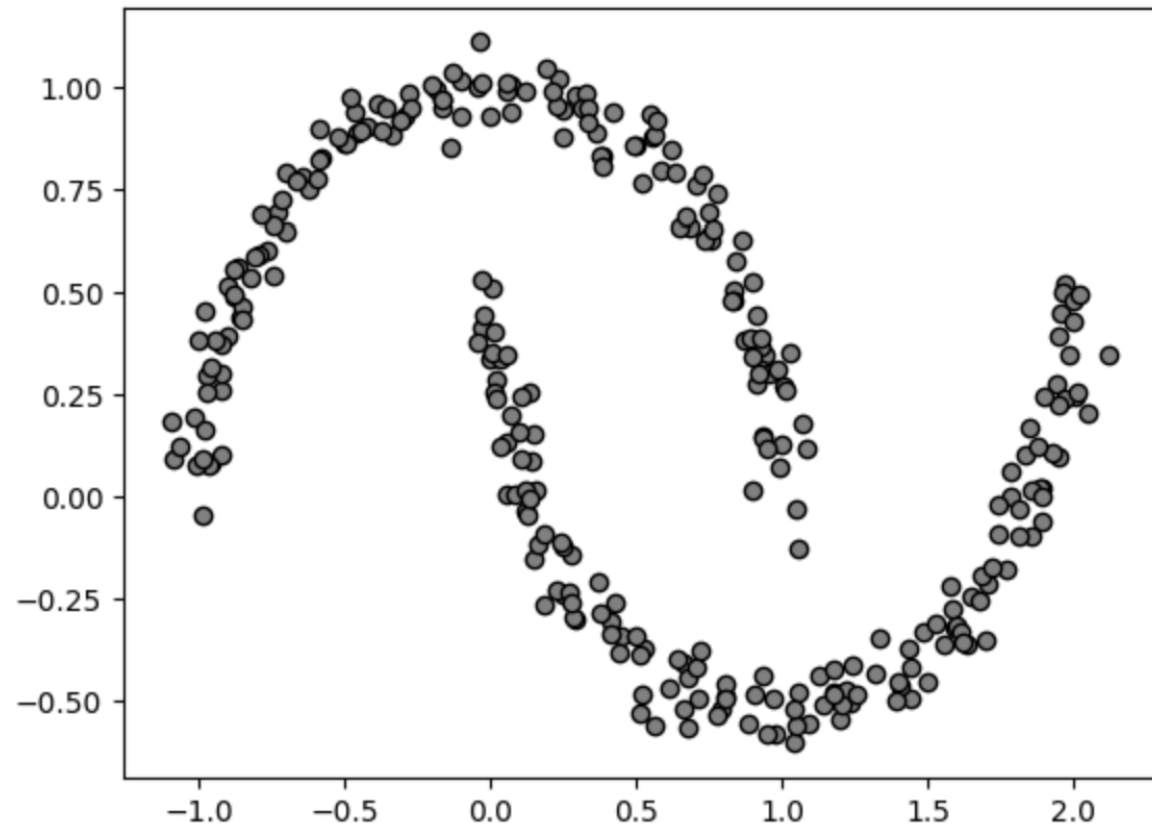
```
1 # 생성된 데이터 차원 확인
2 print(X.shape)
3 print(y.shape)
```

(300, 2)

(300,)

DBSCAN

```
In [3]: ▶ 1 # 라벨링 되지 않은 데이터 플랏  
2 import matplotlib.pyplot as plt  
3 plt.scatter(X[:,0], X[:,1],  
4             c='gray',  
5             edgecolor='black',  
6             marker='o')  
7 plt.show()
```



DBSCAN

In [4]: ▶

```
1 # DB scan
2 from sklearn.cluster import DBSCAN
3 dbs = DBSCAN(eps=0.2)
4 dbs.fit(X)
5 label_dbs = dbs.labels_
6 print(label_dbs)
```

```
[0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 1 0 1 0 0 0 1 1 1 0 1 0 0 0 0 1 1 1 1 0 0 0
 0 0 1 0 1 1 1 1 1 0 1 1 1 1 0 0 1 1 0 0 1 0 0 1 1 1 0 0 0 0 1 1 1 0 0 0 1
 1 0 1 1 0 1 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 1 0 1 0 0 1 0 1 0 0 0 0 0 0
 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 0 1 0 1 1 0 1 0 1 1 1
 1 0 0 1 0 0 0 1 1 1 0 1 0 1 1 0 1 1 1 0 1 0 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1
 1 0 1 0 1 0 0 1 0 1 1 1 0 1 0 0 0 1 1 1 1 0 1 0 1 0 1 1 1 0 1 0 1 0 0 0 1
 1 0 1 0 1 1 0 0 1 0 1 1 0 1 1 1 1 1 0 0 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 0 1
 1 0 0 0 0 0 1 1 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 0 0 1 0 1 1 1 0 0 0 1 0 0
 0 1 1 0]
```

DBSCAN

In [5]: ▶

```
1 # 시각화를 위한 데이터프레임 생성
2 import pandas as pd
3 dbs_columns = ['dbs_comp1', 'dbs_comp2']
4 X_dbs_df = pd.DataFrame(X, columns=dbs_columns)
5 X_dbs_df['target'] = y
6 X_dbs_df['label_dbs'] = label_dbs
7 X_dbs_df.head(5)
```

Out[5]:

	dbs_comp1	dbs_comp2	target	label_dbs
0	0.715413	-0.494089	1	0
1	0.246789	-0.240091	1	0
2	0.943261	0.346800	0	1
3	0.973742	-0.492901	1	0
4	1.239713	-0.411411	1	0

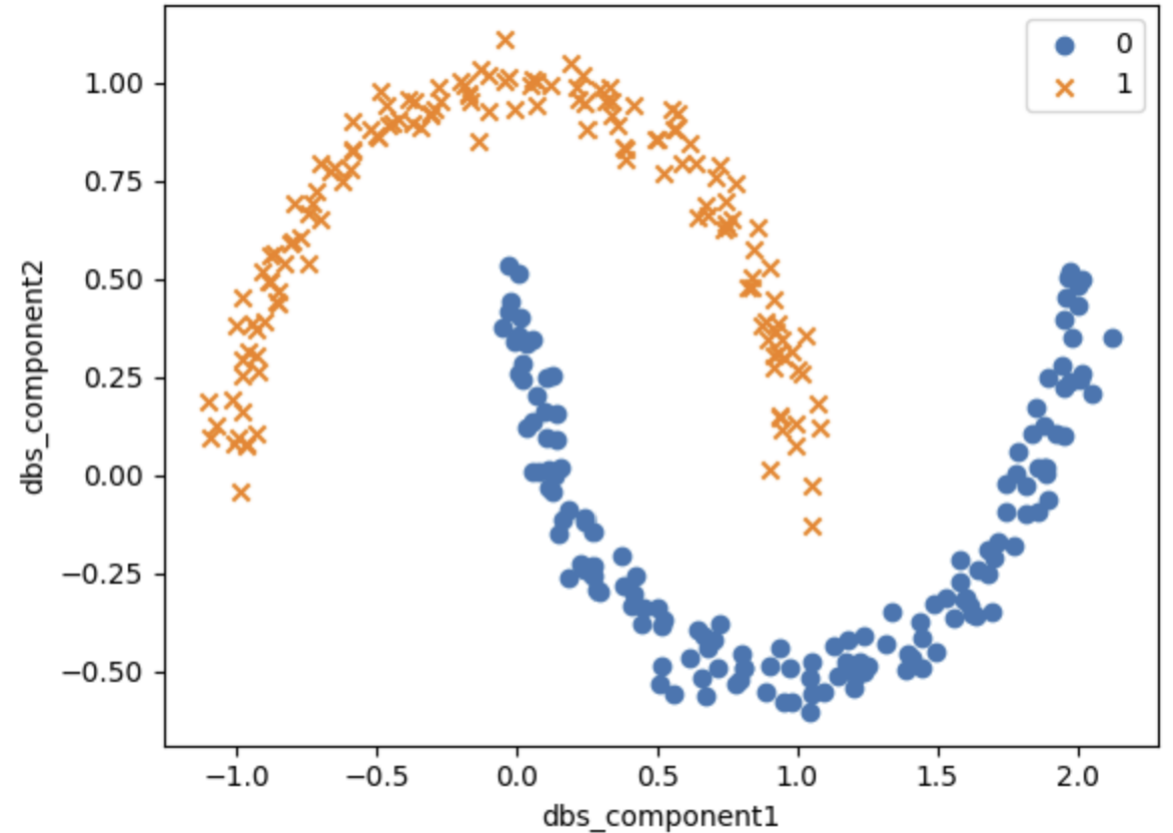
In [6]: ▶

```
1 # 타겟 클래스 종류 확인
2 print(set(X_dbs_df['target']))
3 print(set(X_dbs_df['label_dbs']))
```

```
{0, 1}
{0, 1}
```

DBSCAN

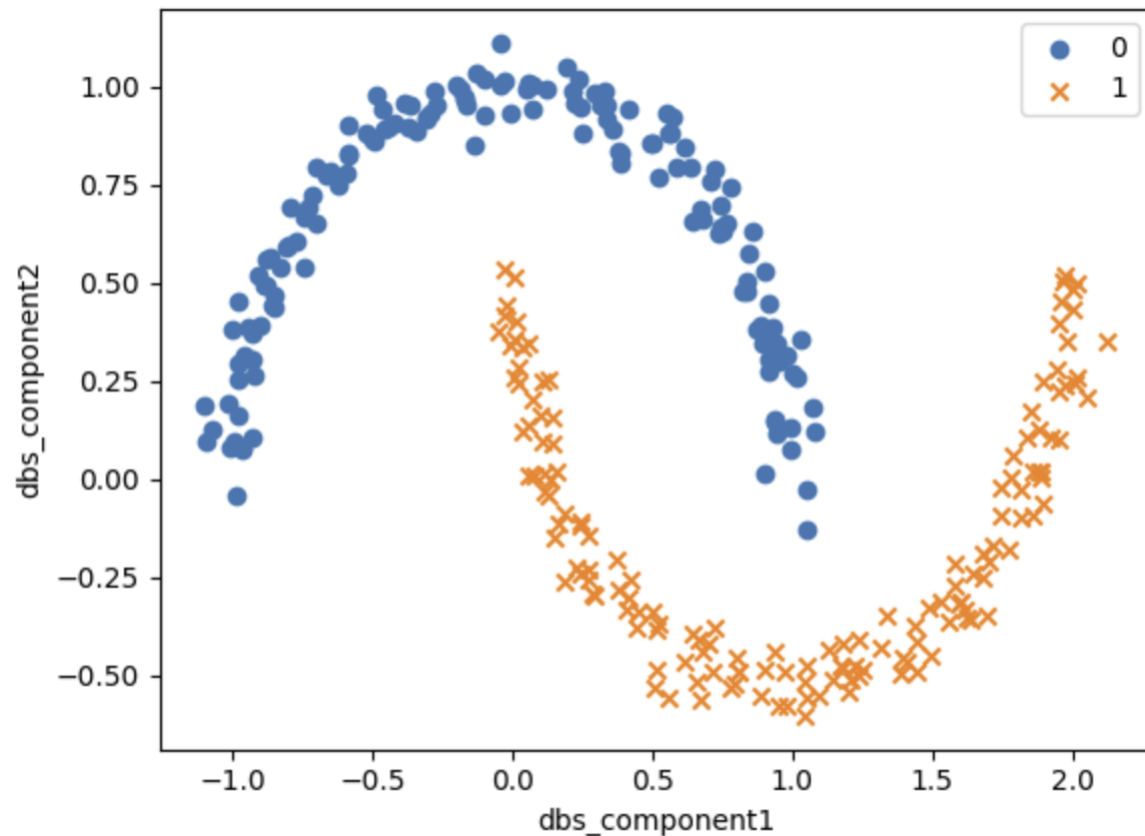
```
In [7]: ▶ 1 # DB scan을 이용한 데이터 플랏
2 df = X_dbs_df
3 markers=['o', 'x']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['label_dbs']==i]
7     target_i = i
8     X1 = df_i['dbs_comp1']
9     X2 = df_i['dbs_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('dbs_component1')
15 plt.ylabel('dbs_component2')
16 plt.legend()
17 plt.show()
```



DBSCAN

In [8]: ▶

```
1 # 실제 타겟 플랏
2 df = X_dbs_df
3 markers=['o','x']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['target']==i]
7     target_i = i
8     X1 = df_i['dbs_comp1']
9     X2 = df_i['dbs_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('dbs_component1')
15 plt.ylabel('dbs_component2')
16 plt.legend()
17 plt.show()
```



DBSCAN

In [9]: ▶

```
1 # 모형 평가
2 from sklearn.metrics import silhouette_score
3 sil_score = silhouette_score(X, label_dbs)
4 print(sil_score)
```

0.3284782012631504

AI/BigData 인텐시브 과정

Section 3. 가우시안 혼합 모형

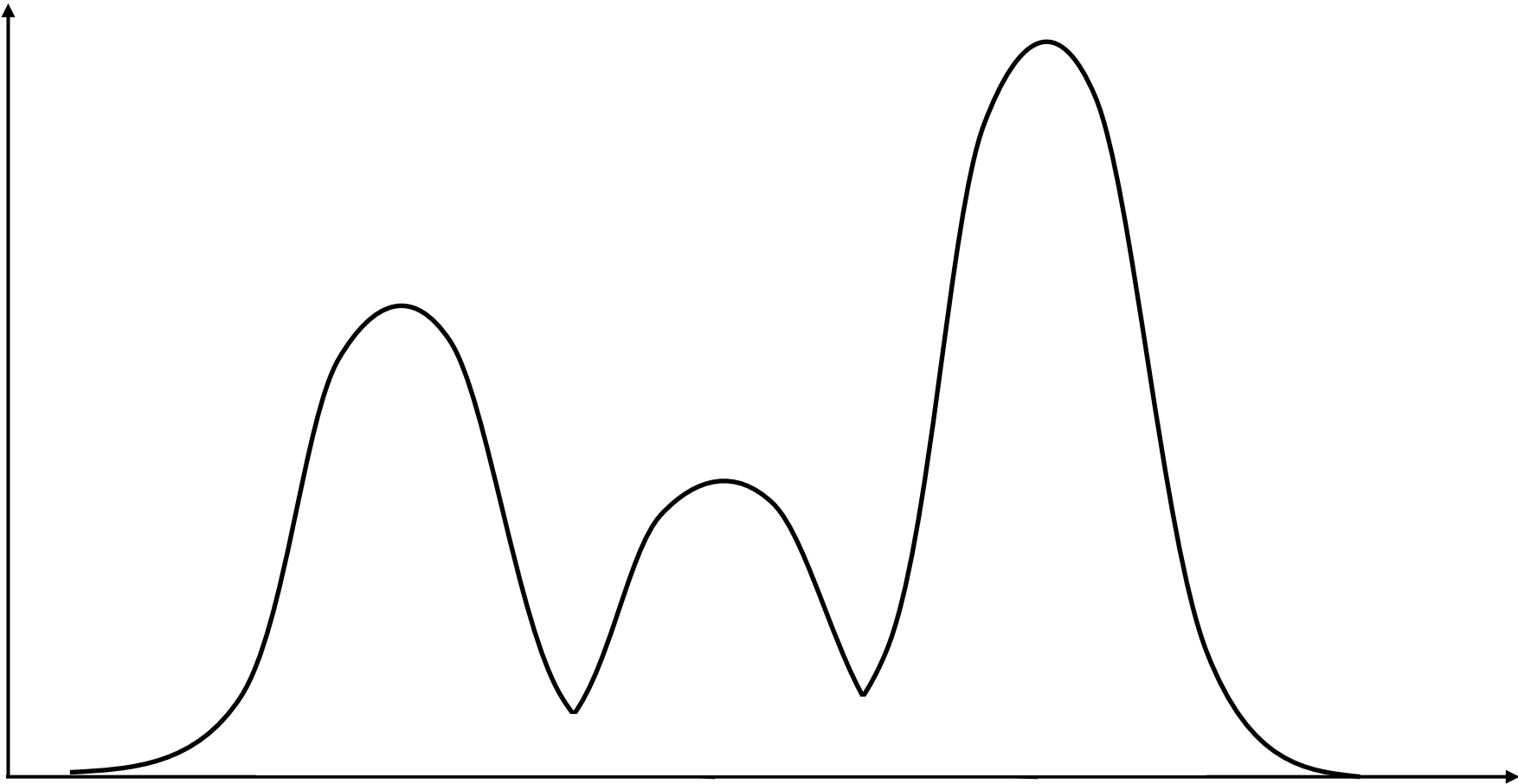
Section 3-1. 가우시안 혼합 모형 개념

가우시안 혼합 모형 개념

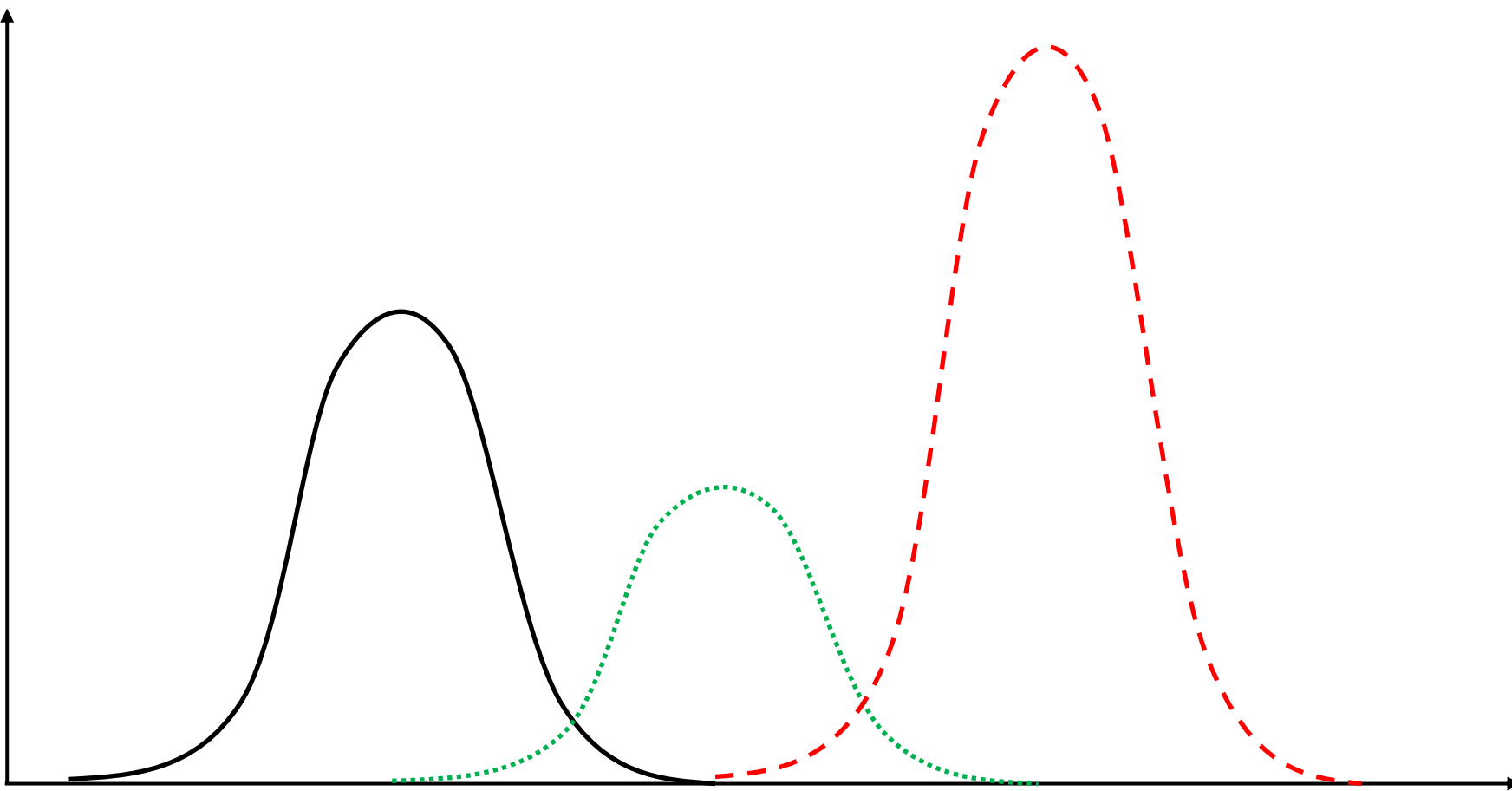
가우시안 혼합 모형(Gaussian Mixture Model)은 전체 집단 안의 하위 집단의 존재를 나타내기 위한 확률 모델

- 가우시안 혼합 모델은 전체 집단의 확률 분포가 가우시안 분포를 따르는 경우를 말함
- 가우시안 혼합 모형은 비지도 학습의 한 종류로 데이터 클러스터링에 사용

가우시안 혼합 모형 개념



가우시안 혼합 모형 개념



AI/BigData 인텐시브 과정

Section 3. 가우시안 혼합 모형

Section 3-1. 가우시안 혼합 모형 실습

가우시안 혼합 모형

In [1]: ▶

```
1 # 데이터 생성
2 from sklearn.datasets import make_blobs
3 X, y = make_blobs(n_samples=200,
4                   n_features=2,
5                   centers=2,
6                   random_state=2)
```

In [2]: ▶

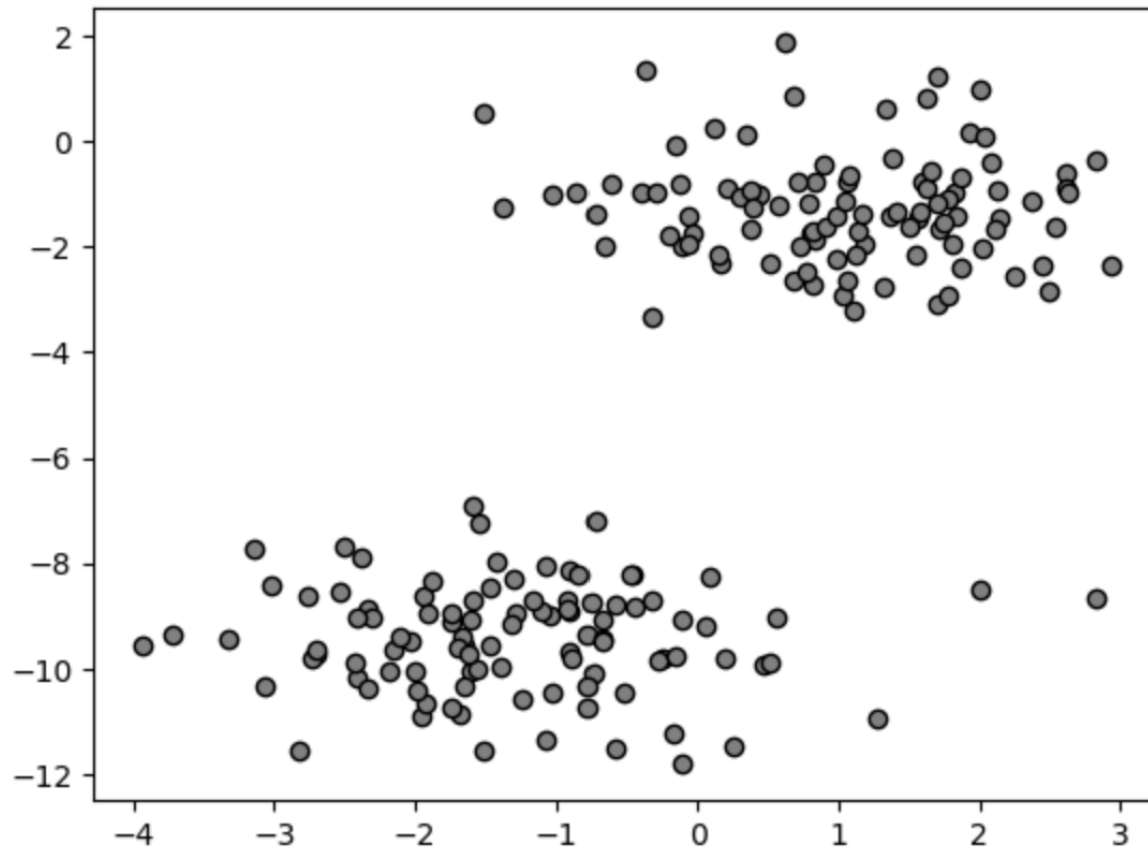
```
1 # 생성된 데이터 차원 확인
2 print(X.shape)
3 print(y.shape)
```

(200, 2)

(200,)

가우시안 혼합 모형

```
In [3]: ▶ 1 # 라벨링 되지 않은 데이터 플랏  
2 import matplotlib.pyplot as plt  
3 plt.scatter(X[:,0], X[:,1],  
4             c='gray',  
5             edgecolor='black',  
6             marker='o')  
7 plt.show()
```



가우시안 혼합 모형

In [4]: ▶

```
1 # 가우시안 혼합 모형
2 from sklearn.mixture import GaussianMixture
3 gmm = GaussianMixture(n_components=2,
4                       random_state=0)
5 gmm.fit(X)
6 label_gmm = gmm.predict(X)
7 print(label_gmm)
```

```
[1 0 1 1 0 1 0 0 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1 1 1 0 1 1 0 0 0 1 0 0 1 0
 1 1 0 1 0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 1 0 1 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1
 0 1 1 1 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 1 0 1 1 1 0 0 1 0 0 1 0 0 0 1 0 0
 1 1 1 1 0 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 0 1 1 0 1 1 0 0 0 0 1
 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 1 0 0 0 0 0 1 1 0 1 0 1 0 1 1
 0 0 1 0 0 1 1 0 1 1 0 0 0 1 0]
```

가우시안 혼합 모형

```
In [5]: ▶ 1 # 시각화를 위한 데이터프레임 생성
2 import pandas as pd
3 gmm_columns = ['gmm_comp1', 'gmm_comp2']
4 X_gmm_df = pd.DataFrame(X, columns=gmm_columns)
5 X_gmm_df['target'] = y
6 X_gmm_df['label_gmm'] = label_gmm
7 X_gmm_df.head(5)
```

Out[5]:

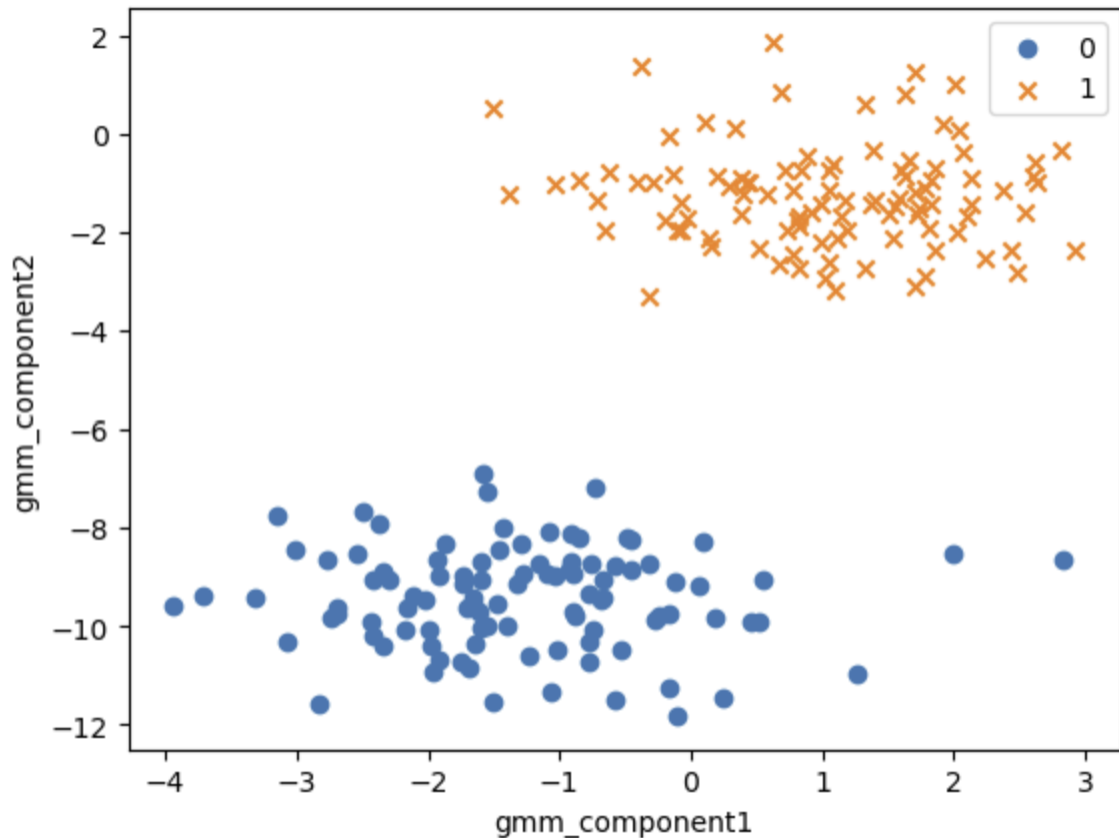
	gmm_comp1	gmm_comp2	target	label_gmm
0	0.381155	-1.644815	1	1
1	-0.583904	-8.784059	0	0
2	1.830865	-1.425235	1	1
3	1.813135	-1.924671	1	1
4	-1.655771	-9.555946	0	0

```
In [6]: ▶ 1 # 타겟 클래스 종류 확인
2 print(set(X_gmm_df['target']))
3 print(set(X_gmm_df['label_gmm']))
```

```
{0, 1}
{0, 1}
```

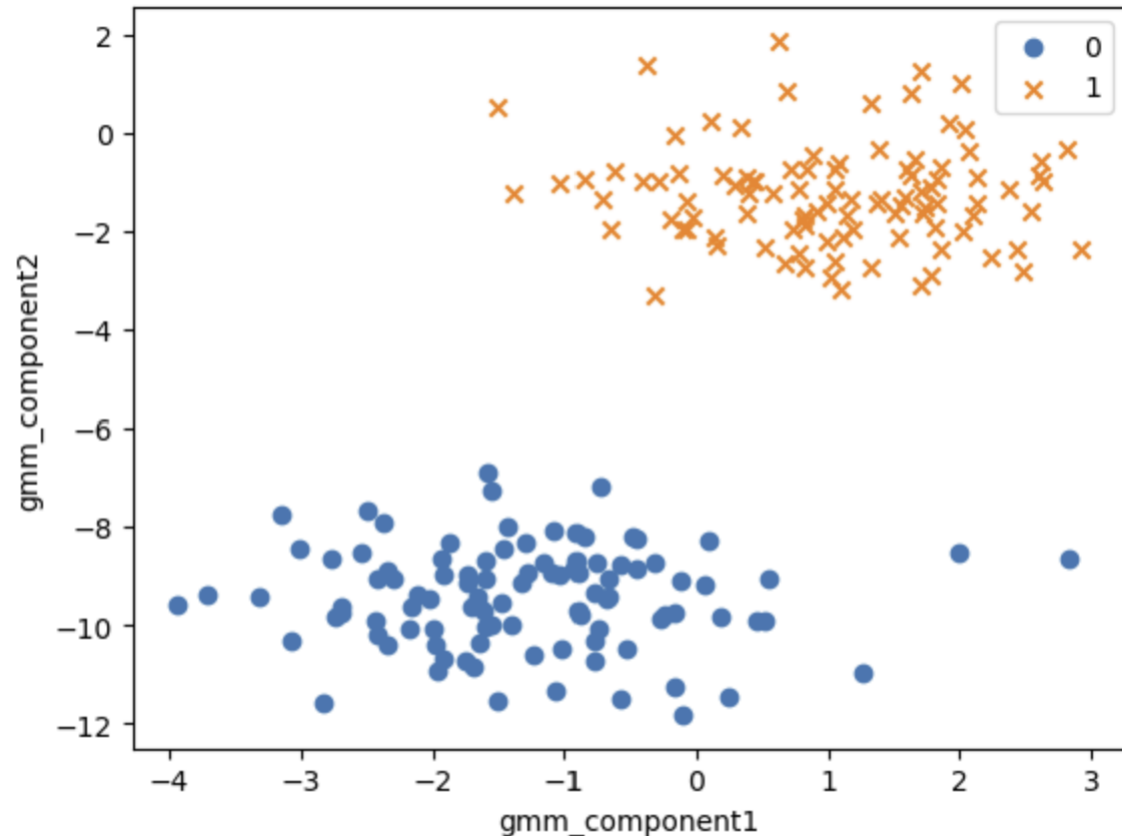

가우시안 혼합 모형

```
In [7]: ▶ 1 # 가우시안 혼합 모형을 이용한 데이터 플랏
2 df = X_gmm_df
3 markers=['o', 'x']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['label_gmm']==i]
7     target_i = i
8     X1 = df_i['gmm_comp1']
9     X2 = df_i['gmm_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('gmm_component1')
15 plt.ylabel('gmm_component2')
16 plt.legend()
17 plt.show()
```



가우시안 혼합 모형

```
In [8]: ▶ 1 # 실제 타겟 플랏
2 df = X_gmm_df
3 markers=['o', 'x']
4
5 for i, mark in enumerate(markers):
6     df_i = df[df['target']==i]
7     target_i = i
8     X1 = df_i['gmm_comp1']
9     X2 = df_i['gmm_comp2']
10    plt.scatter(X1, X2,
11                marker=mark,
12                label=target_i)
13
14 plt.xlabel('gmm_component1')
15 plt.ylabel('gmm_component2')
16 plt.legend()
17 plt.show()
```



가우시안 혼합 모형

In [9]: ▶

```
1 # 모형 평가
2 from sklearn.metrics import silhouette_score
3 sil_score = silhouette_score(X, label_gmm)
4 print(sil_score)
```

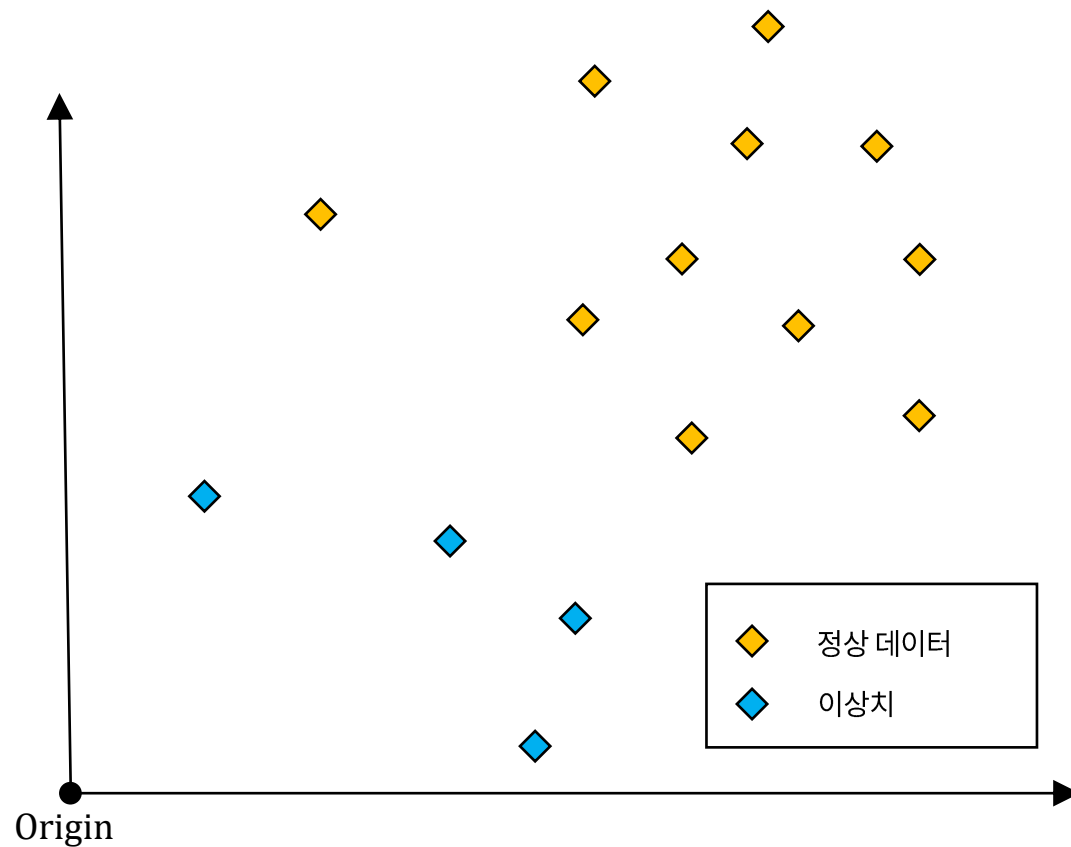
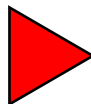
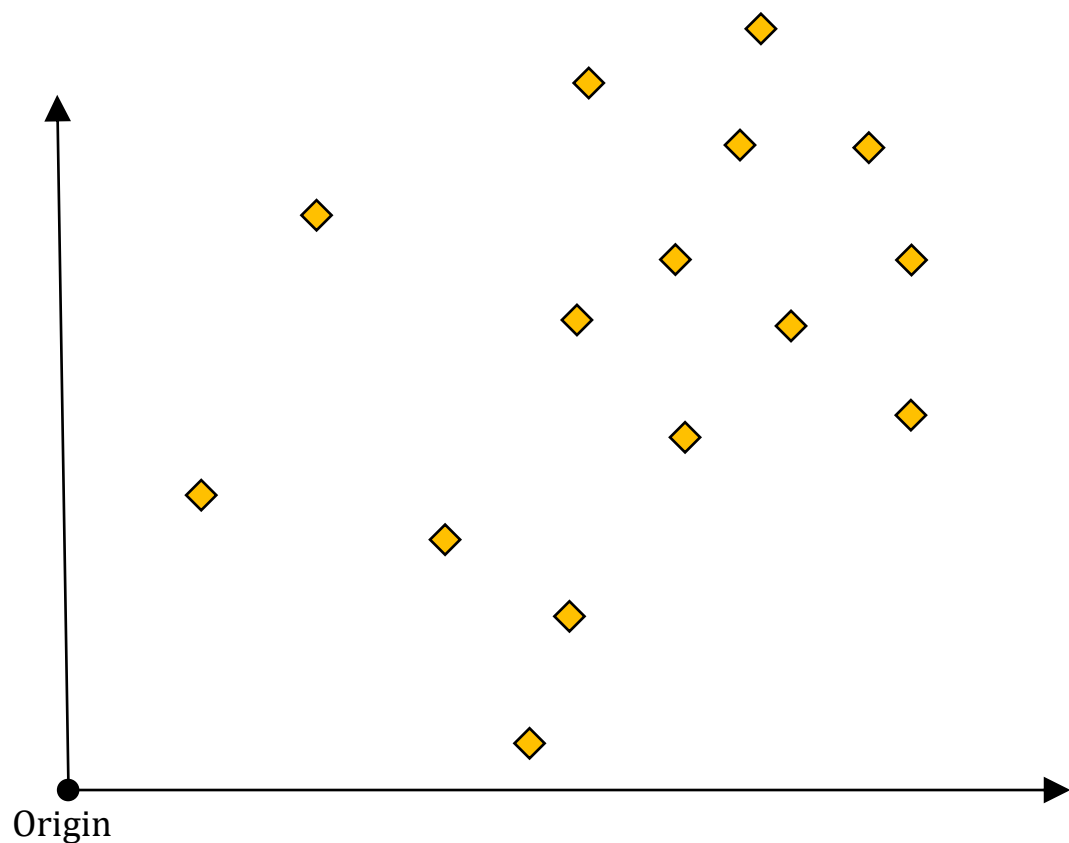
0.7842908753561848

AI/BigData 인텐시브 과정

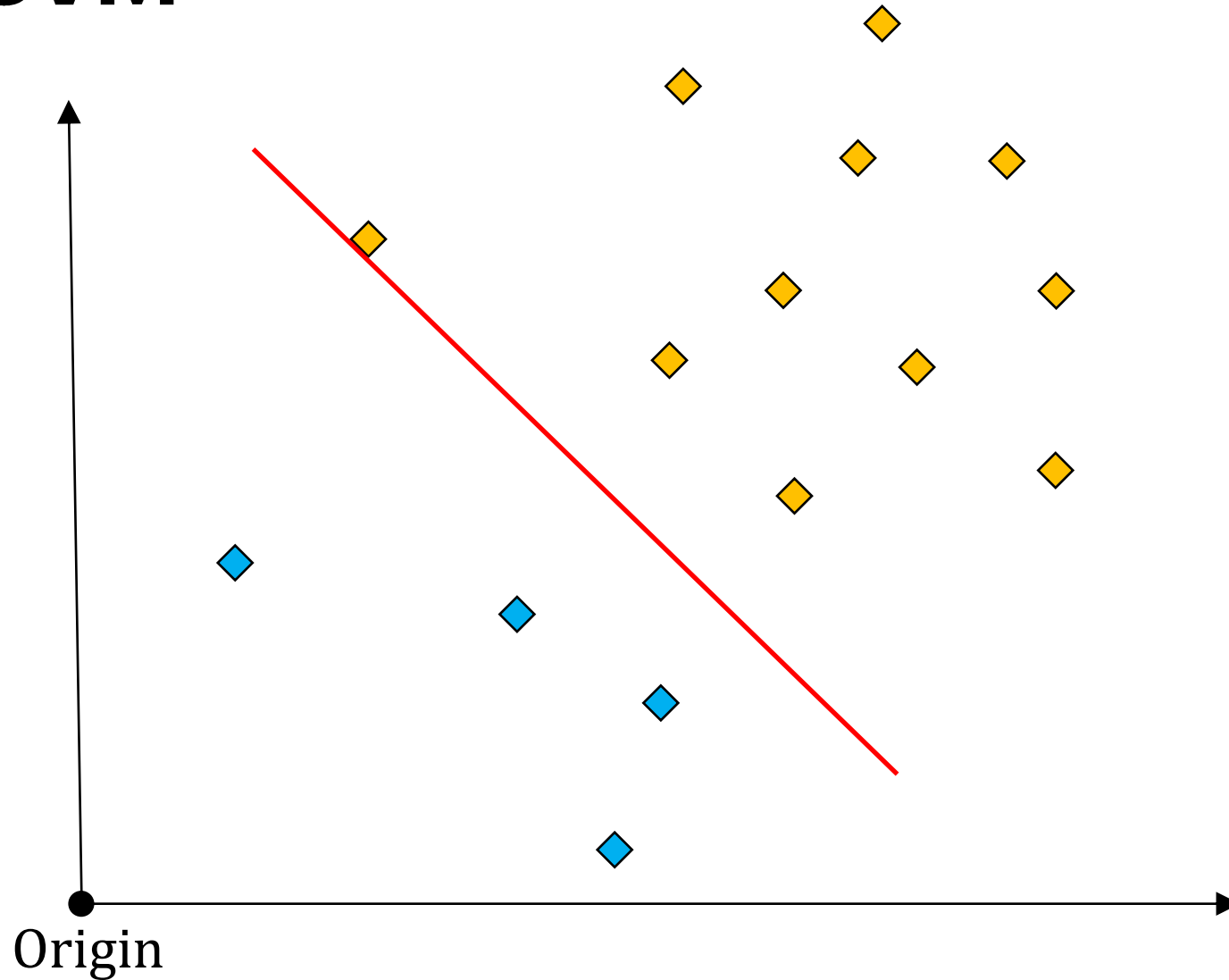
Section 4. One Class SVM

Section 4-1. One Class SVM 개념

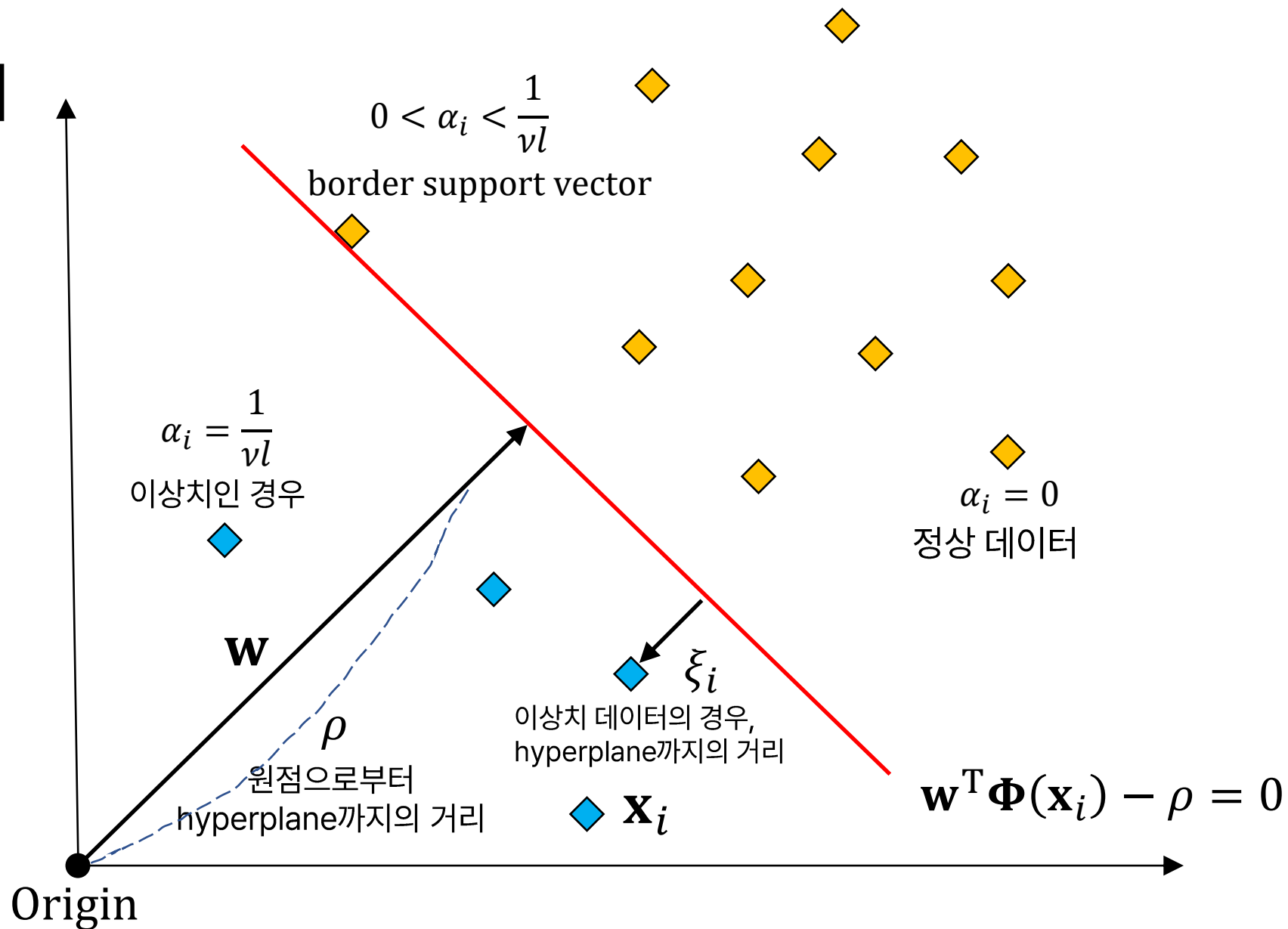
One Class SVM



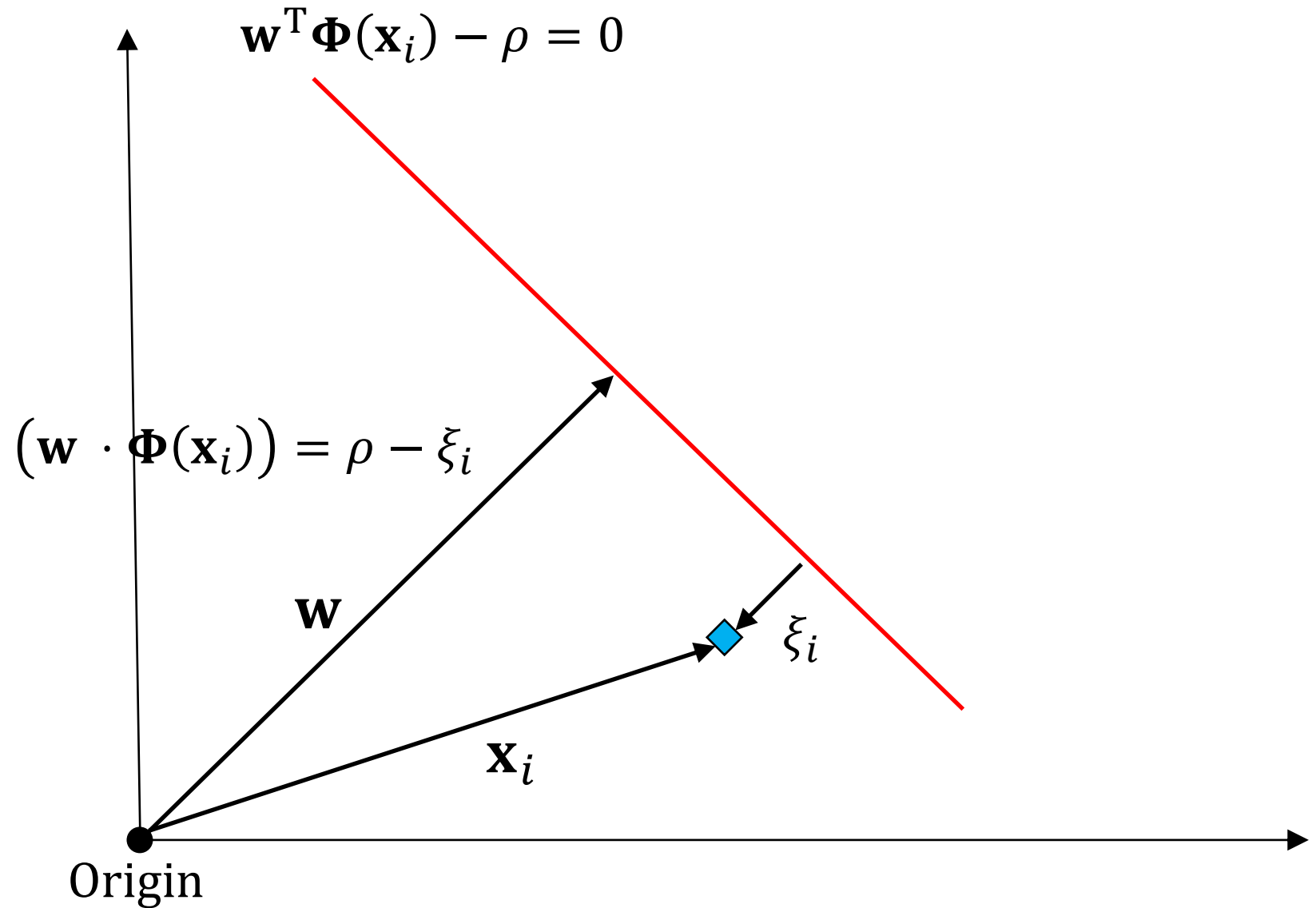
One Class SVM



One Class SVM



One Class SVM



One Class SVM

목적 함수

$$\min_{w, \xi_i, \rho} \frac{1}{2} \|\mathbf{w}\|^2 + \frac{1}{vl} \sum_{i=1}^l \xi_i - \rho$$

subject to

$$\begin{aligned} (\mathbf{w} \cdot \Phi(\mathbf{x}_i)) &\geq \rho - \xi_i && \text{for all } i = 1, 2, \dots, l \\ \xi_i &\geq 0 && \text{for all } i = 1, 2, \dots, l \end{aligned}$$

결정 함수

$$f(\mathbf{x}_i) = \text{sign}(\mathbf{w} \cdot \Phi(\mathbf{x}_i) - \rho)$$

One Class SVM

데이터의 개수

$$\min_{w, \xi_i, \rho} \frac{1}{2} \|w\|^2$$

모델의 변동성 감소

$$+ \frac{1}{\nu l} \sum_{i=1}^l \xi_i$$

이상치 영역에 존재하는
데이터들에 부여되는 패널티이상치 영역에 존재하는
각 데이터 포인트가
hyperplane과 떨어진 거리

$$- \rho$$

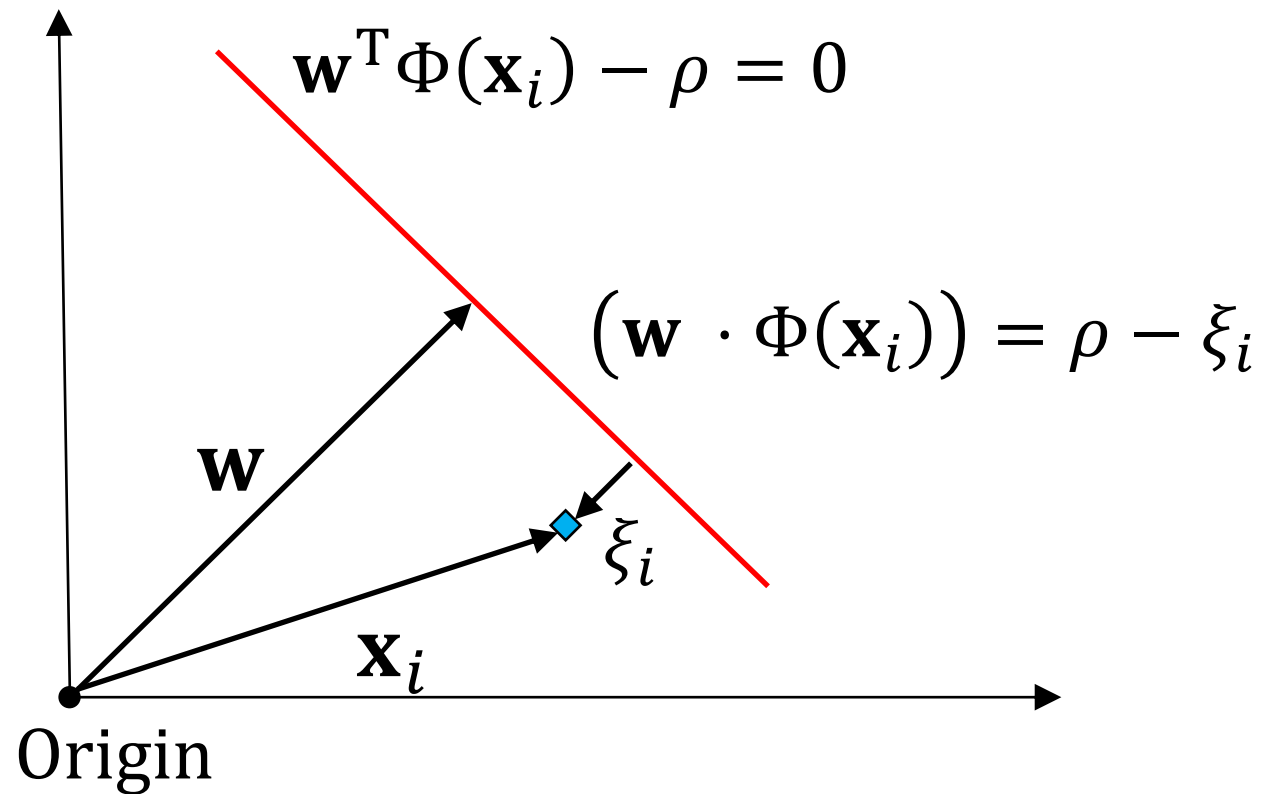
원점과 최대한 멀리 떨어진
hyperplane과의 거리

$$0 \leq \nu \leq 1$$

사용자가 지정하는 하이퍼 파라미터

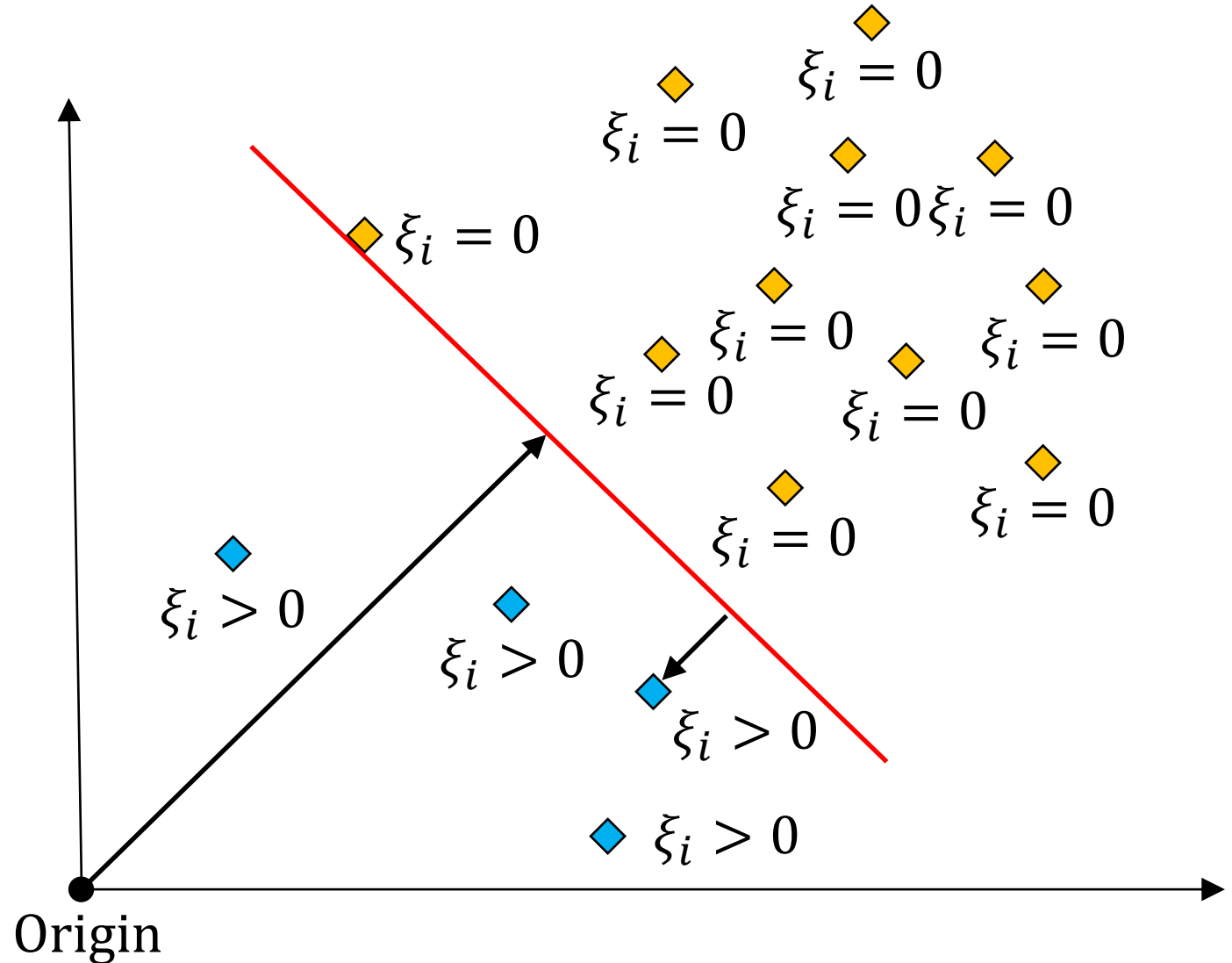
One Class SVM subject to

$$(\mathbf{w} \cdot \Phi(\mathbf{x}_i)) \geq \rho - \xi_i \quad \text{for all } i = 1, 2, \dots, l$$



One Class SVM

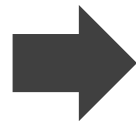
$$\xi_i \geq 0 \quad \text{for all } i = 1, 2, \dots, l$$



One Class SVM

$$L_p = \frac{1}{2} \|\mathbf{w}\|^2 + \frac{1}{\nu l} \sum_{i=1}^l \xi_i - \rho - \underbrace{\sum_{i=1}^l \alpha_i (\mathbf{w} \Phi(\mathbf{x}_i) - \rho + \xi_i)}_{\text{제약식 1}} - \underbrace{\sum_{i=1}^l \beta_i \xi_i}_{\text{제약식 2}}$$

$$\frac{\partial L_p}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^l \alpha_i \Phi(\mathbf{x}_i) = 0$$



$$\therefore \mathbf{w} = \sum_{i=1}^l \alpha_i \Phi(\mathbf{x}_i)$$

$$\frac{\partial L_p}{\partial \xi_i} = \frac{1}{\nu l} - \alpha_i - \beta_i = 0$$



$$\therefore \alpha_i = \frac{1}{\nu l} - \beta_i$$

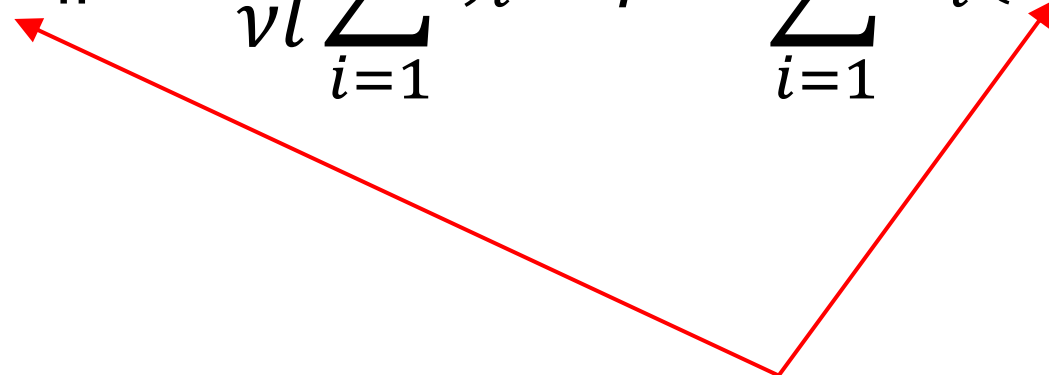
$$\frac{\partial L_p}{\partial \rho} = -1 + \sum_{i=1}^l \alpha_i = 0$$



$$\therefore \sum_{i=1}^l \alpha_i = 1$$

One Class SVM

$$L_p = \frac{1}{2} \|\mathbf{w}\|^2 + \frac{1}{\nu l} \sum_{i=1}^l \xi_i - \rho - \sum_{i=1}^l \alpha_i (\mathbf{w} \Phi(\mathbf{x}_i) - \rho + \xi_i) - \sum_{i=1}^l \beta_i \xi_i$$

$$\mathbf{w} = \sum_{i=1}^l \alpha_i \Phi(\mathbf{x}_i)$$
Two red arrows originate from the equation $\mathbf{w} = \sum_{i=1}^l \alpha_i \Phi(\mathbf{x}_i)$. One arrow points to the $\mathbf{w}$ term in the $\|\mathbf{w}\|^2$ part of the loss function L_p . The other arrow points to the $\mathbf{w}$ term inside the summation $\sum_{i=1}^l \alpha_i (\mathbf{w} \Phi(\mathbf{x}_i) - \rho + \xi_i)$.

Section

$$L_d = \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{red}} + \underbrace{\frac{1}{\nu l} \sum_{i=1}^l \xi_i}_{\text{green}} - \underbrace{\rho}_{\text{purple}} - \underbrace{\sum_{i=1}^l \alpha_i (\mathbf{w} \Phi(\mathbf{x}_i) - \rho + \xi_i)}_{\text{blue}} - \underbrace{\sum_{i=1}^l \beta_i \xi_i}_{\text{yellow}}$$

$$= \underbrace{\frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)}_{\text{red}} + \underbrace{\frac{1}{\nu l} \sum_{i=1}^l \xi_i}_{\text{green}} - \underbrace{\rho}_{\text{purple}} - \underbrace{\sum_{i=1}^l \alpha_i \left(\sum_{j=1}^l \alpha_j \Phi(\mathbf{x}_j) \Phi(\mathbf{x}_j) - \rho + \xi_i \right)}_{\text{blue}} - \underbrace{\sum_{i=1}^l \beta_i \xi_i}_{\text{yellow}}$$

$$= \underbrace{\frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)}_{\text{red}} + \underbrace{\frac{1}{\nu l} \sum_{i=1}^l \xi_i}_{\text{green}} - \underbrace{\rho}_{\text{purple}} - \underbrace{\sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)}_{\text{blue}} + \underbrace{\rho \sum_{i=1}^l \alpha_i}_{\text{cyan}} - \underbrace{\sum_{i=1}^l \alpha_i \xi_i}_{\text{black}} - \underbrace{\sum_{i=1}^l \beta_i \xi_i}_{\text{yellow}}$$

$$= \underbrace{\frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)}_{\text{red}} + \underbrace{\left(\rho \sum_{i=1}^l \alpha_i - \rho \right)}_{\text{cyan}} - \underbrace{\sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)}_{\text{blue}} + \underbrace{\sum_{i=1}^l \left(\frac{1}{\nu l} - \alpha_i - \beta_i \right) \xi_i}_{\text{green, black, yellow}}$$

$$= -\frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j) + \underbrace{\left(\rho \sum_{i=1}^l \alpha_i - \rho \right)}_{\text{cyan, purple}} + \underbrace{\sum_{i=1}^l \left(\frac{1}{\nu l} - \alpha_i - \beta_i \right) \xi_i}_{\text{green, black, yellow}}$$

One Class SVM

$$L_d = -\frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j) + \left(\rho \sum_{i=1}^l \alpha_i - \rho \right) + \sum_{i=1}^l \left(\frac{1}{\nu l} - \alpha_i - \beta_i \right) \xi_i$$

$\because \sum_{i=1}^l \alpha_i = 1$ $\because \alpha_i = \frac{1}{\nu l} - \beta_i$

One Class SVM

$$\min L'_d = \frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l \alpha_i \alpha_j \Phi(\mathbf{x}_i) \Phi(\mathbf{x}_j)$$

$$L'_d = -L_d$$

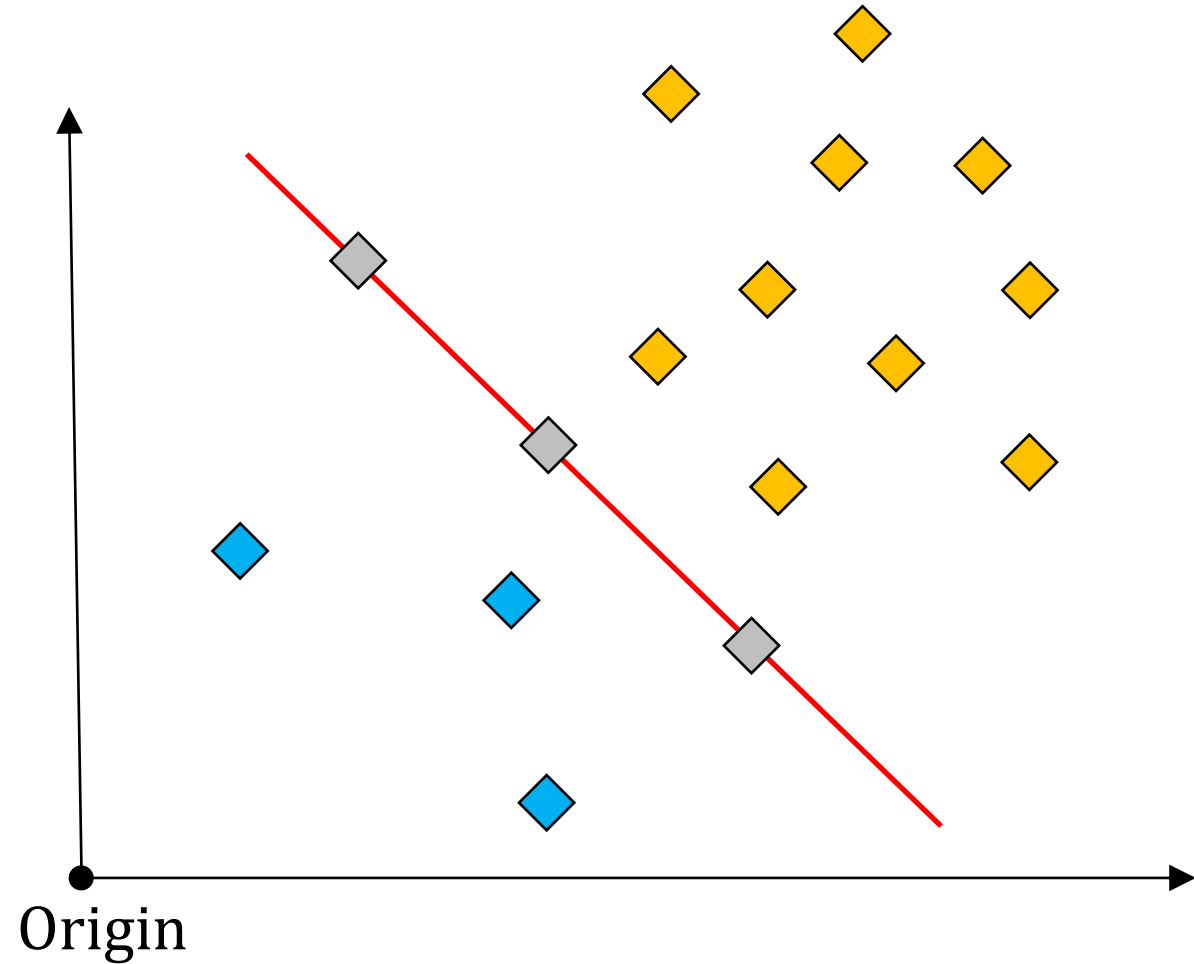
One Class SVM

■ α_i 값 범위에 따른 의미

Case 1)  $\alpha_i = 0$

Case 2)  $\alpha_i = \frac{1}{\nu l}$

Case 3)  $0 < \alpha_i < \frac{1}{\nu l}$



■ α_i 값 범위에 따른 의미

Case 1) ◆ $\alpha_i = 0$

$$\alpha_i = \frac{1}{\nu l} - \beta_i$$

$$\Rightarrow \alpha_i + \beta_i = \frac{1}{\nu l}$$

$$\Rightarrow \beta_i = \frac{1}{\nu l} \neq 0 \quad (\because \alpha_i = 0)$$

$$\Rightarrow \xi_i = 0 \quad (\because \beta_i \xi_i = 0)$$

$\Rightarrow$ 결국 이 객체들은 hyperplane으로 부터 멀리 떨어져 있는 정상 데이터라는 뜻

사용할 조건

$$1) \sum_{i=1}^l \alpha_i = 1$$

$$2) \alpha_i = \frac{1}{\nu l} - \beta_i$$

$$3) \beta_i \xi_i = 0$$

$$4) 0 \leq \alpha_i \leq \frac{1}{\nu l}$$

■ α_i 값 범위에 따른 의미

Case 2) $\blacklozenge \quad \alpha_i = \frac{1}{\nu l}$

$$\alpha_i = \frac{1}{\nu l} - \beta_i$$

$$\Rightarrow \alpha_i + \beta_i = \frac{1}{\nu l}$$

$$\Rightarrow \beta_i = 0 \quad \left(\because \alpha_i = \frac{1}{\nu l} \right)$$

$$\Rightarrow \xi_i > 0 \quad (\because \beta_i \xi_i = 0)$$

$\Rightarrow$ 결국 이 객체들은 hyperplane으로부터 안쪽에 있는 이상치 데이터라는 뜻

사용할 조건

$$1) \quad \sum_{i=1}^l \alpha_i = 1$$

$$2) \quad \alpha_i = \frac{1}{\nu l} - \beta_i$$

$$3) \quad \beta_i \xi_i = 0$$

$$4) \quad 0 \leq \alpha_i \leq \frac{1}{\nu l}$$

■ α_i 값 범위에 따른 의미

Case 3) $\diamond \quad 0 < \alpha_i < \frac{1}{vl}$

$$\alpha_i = \frac{1}{vl} - \beta_i$$

$$\Rightarrow \alpha_i + \beta_i = \frac{1}{vl}$$

$$\Rightarrow \beta_i > 0 \quad \left(\because 0 < \alpha_i < \frac{1}{vl} \right)$$

$$\Rightarrow \xi_i = 0 \quad (\because \beta_i \xi_i = 0)$$

$\Rightarrow \alpha_i \neq 0, \xi_i = 0$ 이므로 결국 이 객체들은 hyperplane 위에 존재하는 데이터라는 뜻

사용할 조건

$$1) \quad \sum_{i=1}^l \alpha_i = 1$$

$$2) \quad \alpha_i = \frac{1}{vl} - \beta_i$$

$$3) \quad \beta_i \xi_i = 0$$

$$4) \quad 0 \leq \alpha_i \leq \frac{1}{vl}$$

■ ν 값의 역할

▶ α_i 가 가질 수 있는 최댓값 $\alpha_i = \frac{1}{\nu l} \quad \left(\because 0 \leq \alpha_i \leq \frac{1}{\nu l} \right)$

▶ $\sum_{i=1}^l \alpha_i = 1$ 이므로, 최소 $\alpha_i = \frac{1}{\nu l}$ 인 서포트 벡터 νl 개 필요 $\left(\frac{1}{\nu l} \times \nu l = 1 \right)$

예를 들어, $l = 1000, \nu = 0.1$ 이면,

$$\alpha_i = \frac{1}{\nu l} = \frac{1}{0.1 \times 1000} = \frac{1}{100} \text{ 이고, } \sum_{i=1}^l \alpha_i = 1 \text{ 이므로}$$

$$\alpha_i = \frac{1}{100} \text{ 인 서포트 벡터 100개 필요 } \left(\frac{1}{100} \times 100 = 1 \right)$$

즉, $l = 1000$ 이므로 전체 데이터 1000개 중

$$\alpha_i = \frac{1}{100} \text{ 인 서포트 벡터 100개, } \alpha_i = 0 \text{ 인 서포트 벡터 900개}$$

AI/BigData 인텐시브 과정

Section 4. One Class SVM

Section 4-2. One Class SVM 실습

One Class SVM

```
In [1]: 1 import pandas as pd
        2 from sklearn import svm
        3 from sklearn.metrics import silhouette_score
```

```
In [2]: 1 df = pd.read_csv("./data/wine_data.csv")
        2 df
```

Out[2]:

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

178 rows × 14 columns

One Class SVM

In [3]: 1 X = df[['Alcohol']] One Class SVM 사용하려면 대괄호 2개 사용

In [4]: 1 clf = svm.OneClassSVM(gamma='auto', nu=0.5)
2 clf.fit(X)

Out [4]:
OneClassSVM
OneClassSVM(gamma='auto')

In [5]: 1 pred_svm = clf.predict(X)
2 print(pred_svm)

```
[ -1  1  1 -1  1 -1 -1 -1 -1 -1 -1  1 -1 -1  1 -1 -1 -1  1 -1 -1  1 -1
  1 -1  1  1 -1 -1  1  1  1  1  1  1  1 -1 -1 -1  1  1 -1  1 -1 -1 -1
 -1 -1 -1 -1 -1  1  1  1 -1  1  1  1  1  1  1 -1  1 -1  1  1 -1  1 -1
  1 -1 -1 -1 -1 -1  1  1 -1  1 -1 -1 -1  1 -1 -1 -1 -1 -1  1  1 -1  1
 -1  1  1  1 -1  1  1 -1  1  1  1  1 -1 -1 -1  1 -1 -1 -1 -1  1  1 -1
 -1 -1  1 -1 -1 -1  1 -1  1 -1 -1 -1  1  1  1  1  1  1 -1 -1  1  1  1
  1  1 -1 -1  1 -1  1  1 -1  1  1  1 -1  1 -1  1  1  1 -1 -1 -1  1  1
  1  1 -1  1 -1  1  1  1  1 -1]
```

In [6]: 1 # 모형 평가
2 sil_score = silhouette_score(X, pred_svm)
3 print(sil_score)

0.0986864660690129

One Class SVM

```
In [7]: 1 X2 = df[['Alcohol', 'Magesium']]
```

```
In [8]: 1 clf2 = svm.OneClassSVM(gamma='auto', nu=0.5)
        2 clf2.fit(X2)
```

```
Out[8]:
OneClassSVM
OneClassSVM(gamma='auto')
```

```
In [9]: 1 pred_svm2 = clf2.predict(X2)
        2 print(pred_svm2)

[-1  1  1 -1 -1 -1 -1 -1 -1 -1 -1 -1  1 -1  1  1 -1 -1 -1  1 -1  1  1 -1
  1 -1  1  1 -1  1  1  1  1 -1  1 -1 -1  1  1 -1 -1  1  1  1  1 -1  1  1
 -1 -1  1 -1  1 -1  1  1 -1  1  1  1  1  1 -1  1 -1  1 -1  1  1 -1  1 -1
 -1 -1 -1 -1  1 -1 -1  1  1  1 -1  1 -1 -1 -1 -1 -1 -1 -1  1  1  1 -1 -1
 -1  1  1  1  1  1  1 -1  1  1  1  1 -1 -1 -1 -1  1 -1 -1 -1 -1  1 -1
 -1 -1 -1  1 -1  1  1 -1  1 -1 -1  1  1 -1  1  1 -1  1  1  1 -1  1  1 -1
 -1  1 -1  1  1 -1 -1  1 -1  1  1  1 -1  1 -1  1  1  1  1  1 -1 -1  1  1
 -1  1 -1  1  1  1  1 -1 -1  1]
```

```
In [10]: 1 # 모형 평가
        2 sil_score2 = silhouette_score(X2, pred_svm2)
        3 print(sil_score2)
```

```
0.0683328829713904
```

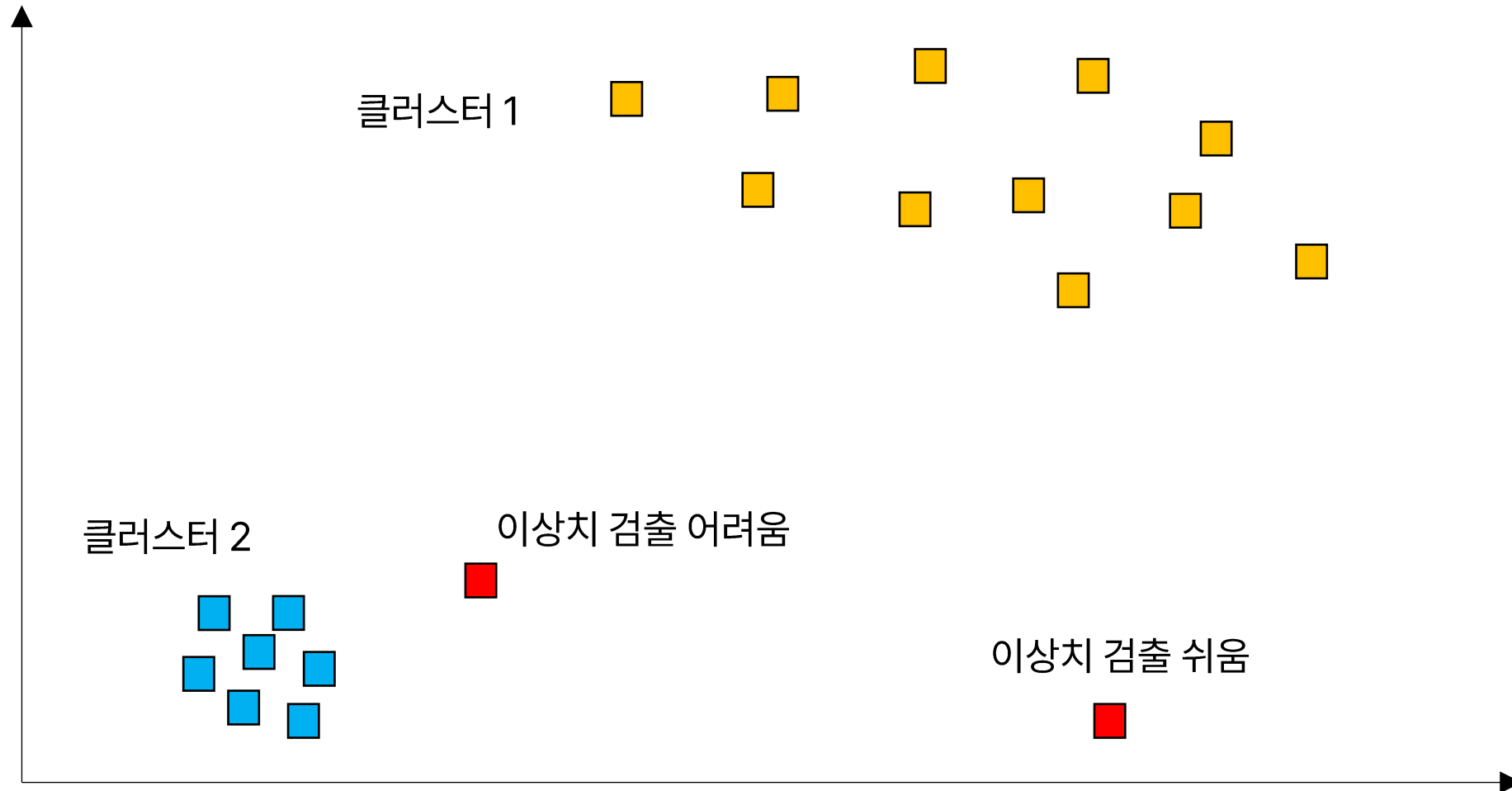
AI/BigData 인텐시브 과정

Section 5. Local Outlier Factor

Section 5-1. Local Outlier Factor 개념

Local Outlier Factor

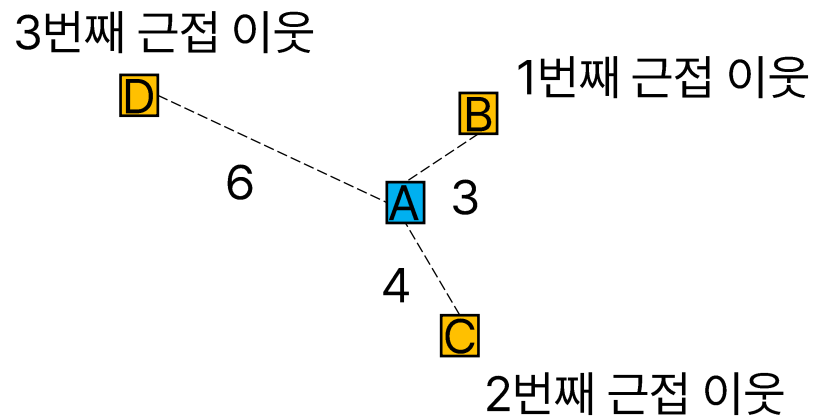
- neighbor들의 밀도(dense)를 고려해 비교



k_distance

$k_distance(A)$: 데이터 A로부터 k번째 근접 이웃까지의 거리

$N_k(A)$: k_distance 안에 들어오는 데이터 집합



$$3_distance(A)=6$$

$$N_3(A) = \{B, C, D\}$$

거리가 6보다 작은 데이터

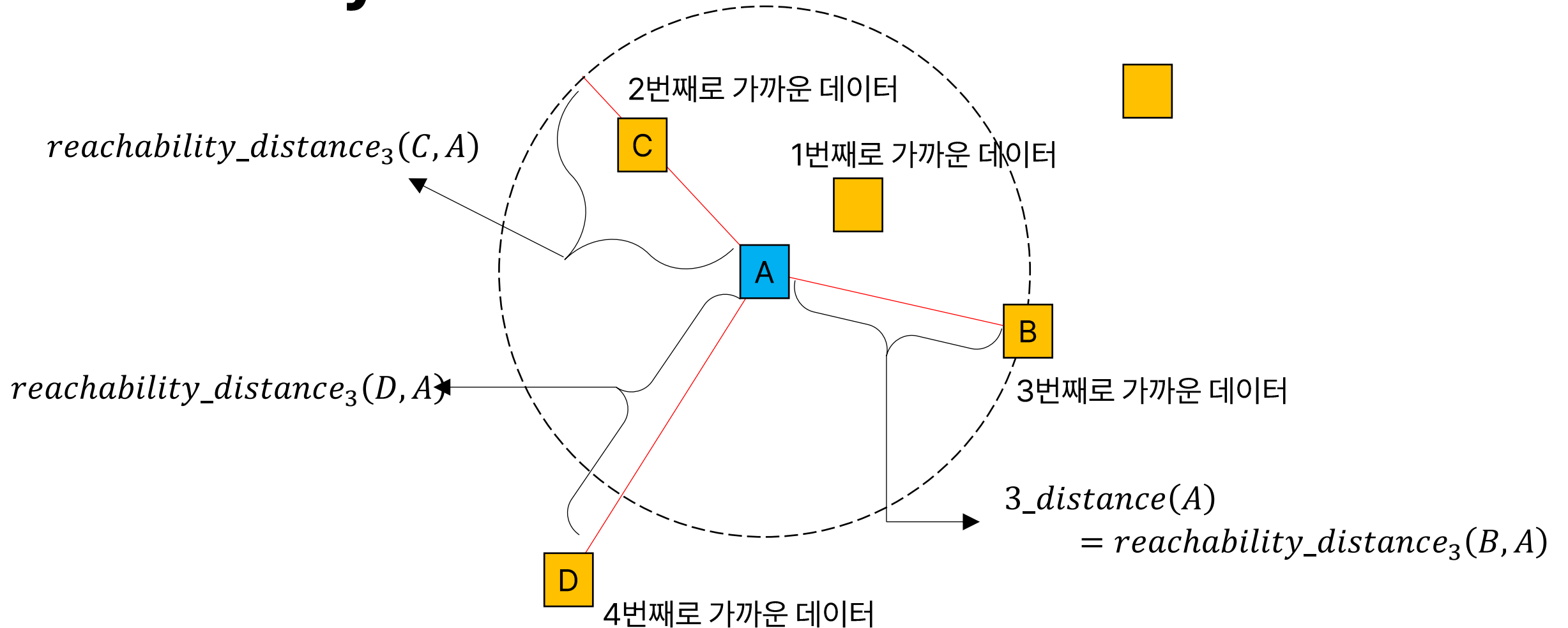
reachability distance

$$\text{reachability_distance}_k(A, B) = \max\{\text{k_distance(B)}, \text{dist(A, B)}\}$$

데이터 B로부터의 k번째 근접 이웃까지의 거리

데이터 A와 데이터 B 사이의 거리

reachability distance

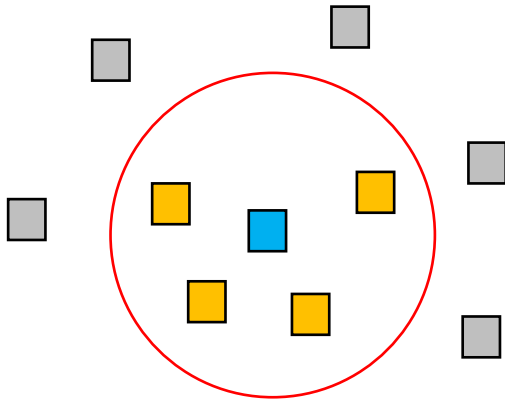


local reachability density(lrd)

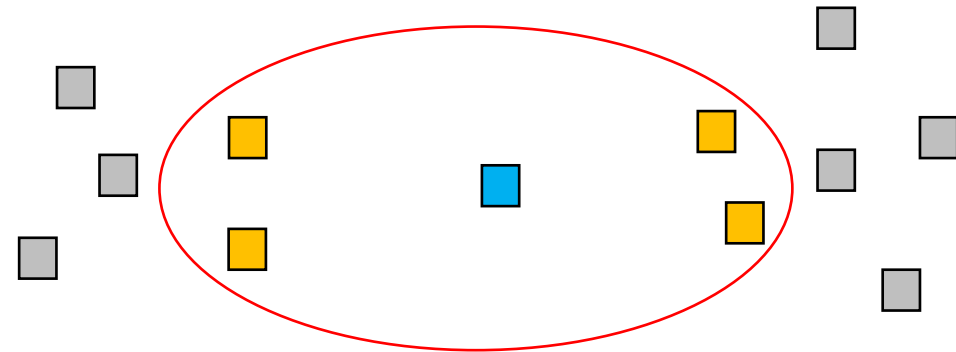
- 기준 데이터 근방 k개의 데이터와의 밀집 정도 측정

$$lrd_k(A) = \frac{|N_k(A)|}{\sum_{O \in N_k(A)} reachability_distance_k(A, O)}$$

$|N_k(A)|$ $\rightarrow$ k_distance안의 데이터 개수



데이터 밀도가 높으므로 분모가 작아짐에 따라
lrd(A) 값이 커짐



데이터 밀도가 낮으므로 분모가 커짐에 따라
lrd(A) 값이 작아짐

local outlier factor

$$LOF_k(A) = \frac{\sum_{B \in N_k(A)} \frac{lrd_k(B)}{lrd_k(A)}}{|N_k(A)|} = \frac{1}{|N_k(A)|} \left\{ \frac{1}{lrd_k(A)} \sum_{B \in N_k(A)} lrd_k(B) \right\}$$

- 데이터 A에 대한 이상치 정도
- 1보다 큰 값을 이상치로 분류, 값이 클수록 아웃라이어 정도가 심함

AI/BigData 인텐시브 과정

Section 5. Local Outlier Factor

Section 5-2. Local Outlier Factor 실습

Local Outlier Factor

```
In [1]: 1 import pandas as pd
        2 from sklearn.neighbors import LocalOutlierFactor
        3 from sklearn.metrics import silhouette_score
```

```
In [2]: 1 df = pd.read_csv("../data/wine_data.csv")
        2 df
```

```
Out[2]:
```

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

178 rows × 14 columns

Local Outlier Factor

```
In [3]: 1 X = df[['Alcohol', 'Magesium']]
```

```
In [4]: 1 clf = LocalOutlierFactor(n_neighbors=3)
2 pred_svm = clf.fit_predict(X)
3 print(pred_svm)
```

Local Outlier Factor에서는 fit만 단독으로 사용하지 않고
fit_predict 사용

```
[ 1  1 -1  1  1  1  1  1  1 -1  1  1  1  1 -1  1  1  1  1  1  1  1  1
 1  1  1  1  1  1  1  1  1  1  1 -1  1  1  1  1  1  1  1  1 -1  1
 1  1  1  1  1  1  1  1  1  1  1  1  1  1 -1  1 -1 -1 -1  1 -1  1 -1
 1  1  1  1  1  1  1  1  1  1  1 -1 -1  1 -1 -1 -1 -1  1  1 -1  1 -1
 1  1  1 -1  1 -1 -1  1  1  1  1 -1  1  1 -1 -1  1 -1 -1 -1  1  1  1 -1
 1 -1 -1  1 -1  1 -1  1  1  1  1  1  1  1  1 -1  1 -1  1  1  1 -1  1
 1  1 -1  1  1  1  1  1  1  1  1  1  1 -1  1  1  1  1 -1 -1  1 -1
 1  1  1  1  1  1  1  1  1  1]
```

```
In [5]: 1 # 모형 평가
2 sil_score = silhouette_score(X, pred_svm)
3 print(sil_score)
```

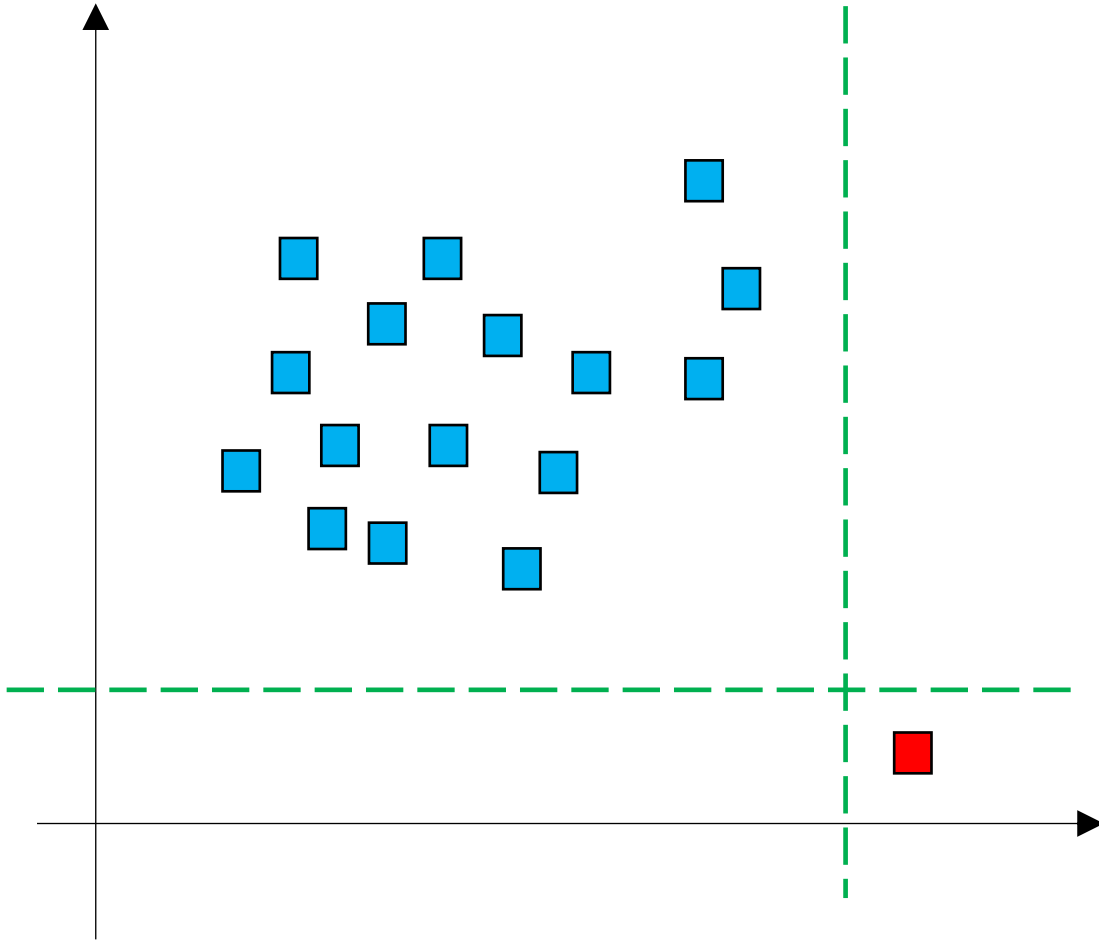
```
0.11619572405864302
```

AI/BigData 인텐시브 과정

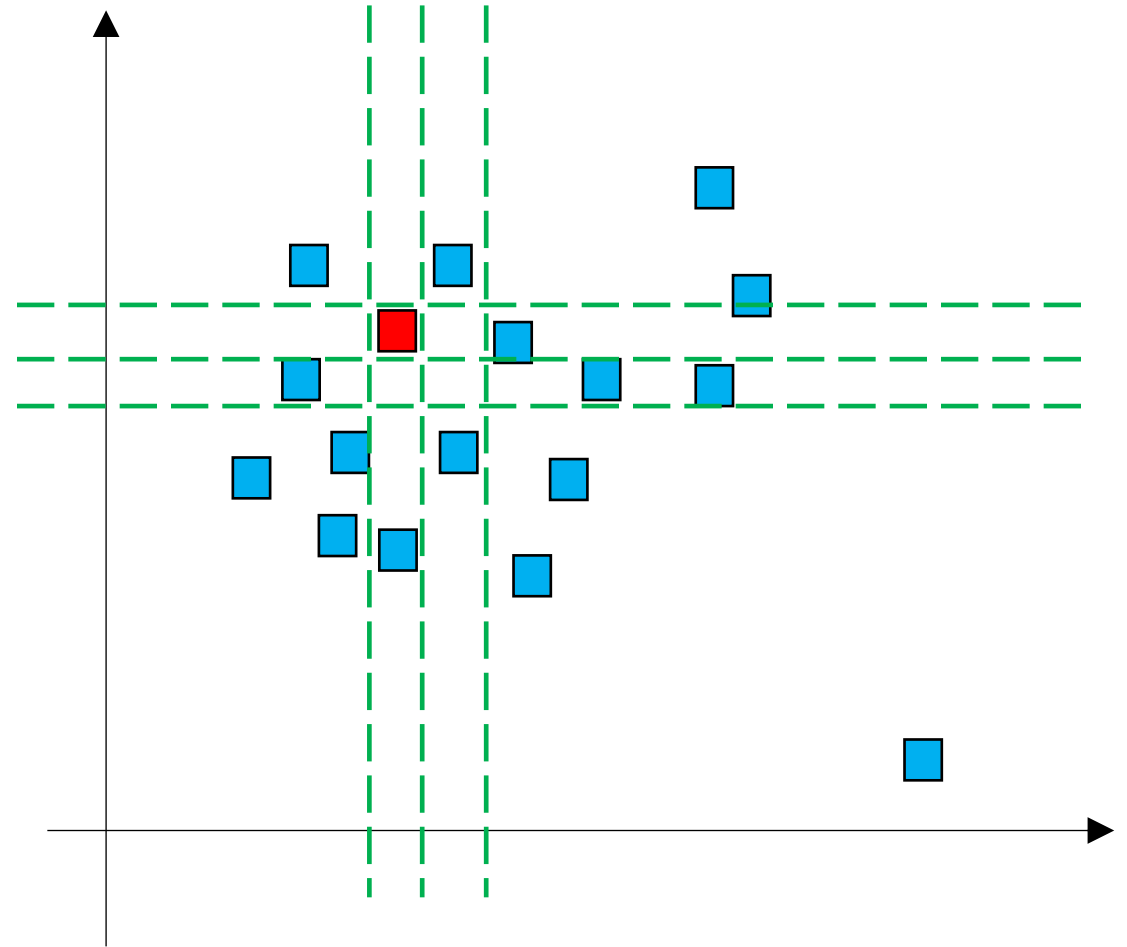
Section 6. Isolation Forest

Section 6-1. Isolation Forest 개념

Isolation Forest



이상치는 쉽게 고립시킬 수 있다.



정상 데이터는 고립시키기 어려움

AI/BigData 인텐시브 과정

Section 6. Isolation Forest

Section 6-2. Isolation Forest 실습

Isolation Forest

```
In [1]: 1 import pandas as pd
        2 from sklearn.ensemble import IsolationForest
        3 from sklearn.metrics import silhouette_score
```

```
In [2]: 1 df = pd.read_csv("../data/wine_data.csv")
        2 df
```

Out[2]:

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

178 rows × 14 columns

Isolation Forest

```
In [3]: 1 X = df[['Alcohol', 'Magesium']]
```

```
In [4]: 1 clf = IsolationForest(random_state=0)
        2 clf.fit(X)
```

```
Out[4]: IsolationForest
IsolationForest(random_state=0)
```

```
In [5]: 1 pred_svm = clf.predict(X)
        2 print(pred_svm)
```

```
[-1  1  1 -1  1  1  1 -1 -1  1  1  1  1 -1  1  1 -1  1  1  1 -1  1  1  1
  1 -1  1  1  1  1  1  1  1 -1  1  1  1  1  1 -1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1 -1  1  1  1 -1  1  1
  1 -1  1  1  1  1 -1 -1  1  1  1  1  1  1  1  1  1 -1  1  1  1  1  1 -1
 -1  1  1  1  1  1  1  1  1  1  1  1  1  1 -1  1  1 -1  1 -1  1  1  1  1
 -1 -1  1  1 -1  1  1  1  1  1  1 -1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1 -1  1  1  1 -1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1]
```

```
In [6]: 1 # 모형 평가
        2 sil_score = silhouette_score(X, pred_svm)
        3 print(sil_score)
```

```
0.42334805508046636
```

AI/BigData 인텐시브 과정

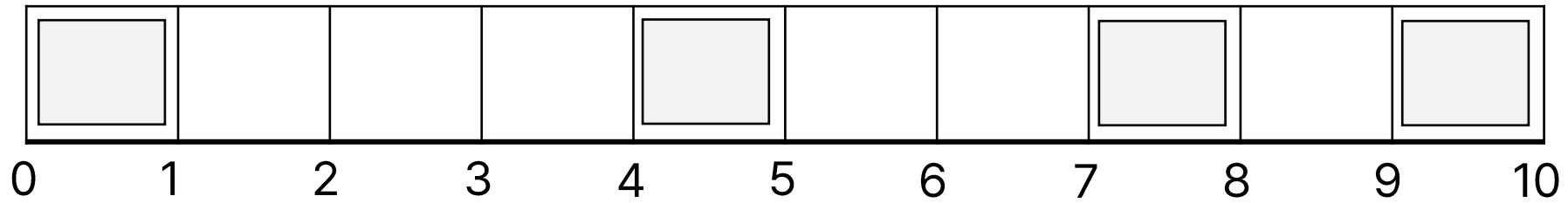
Section 7. 주성분 분석

Section 7-1. 주성분 분석개념

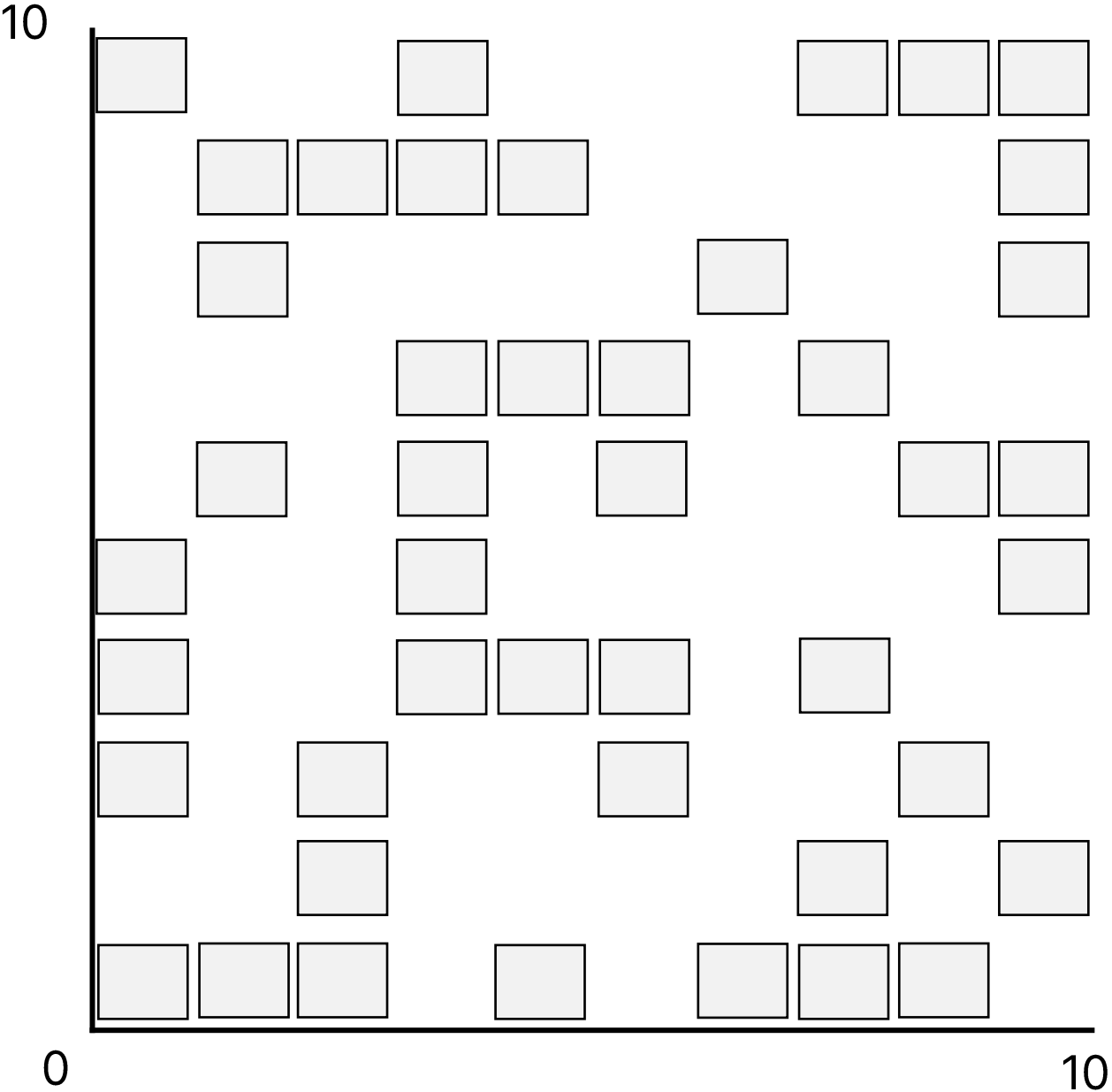
차원 축소(dimension reduction)



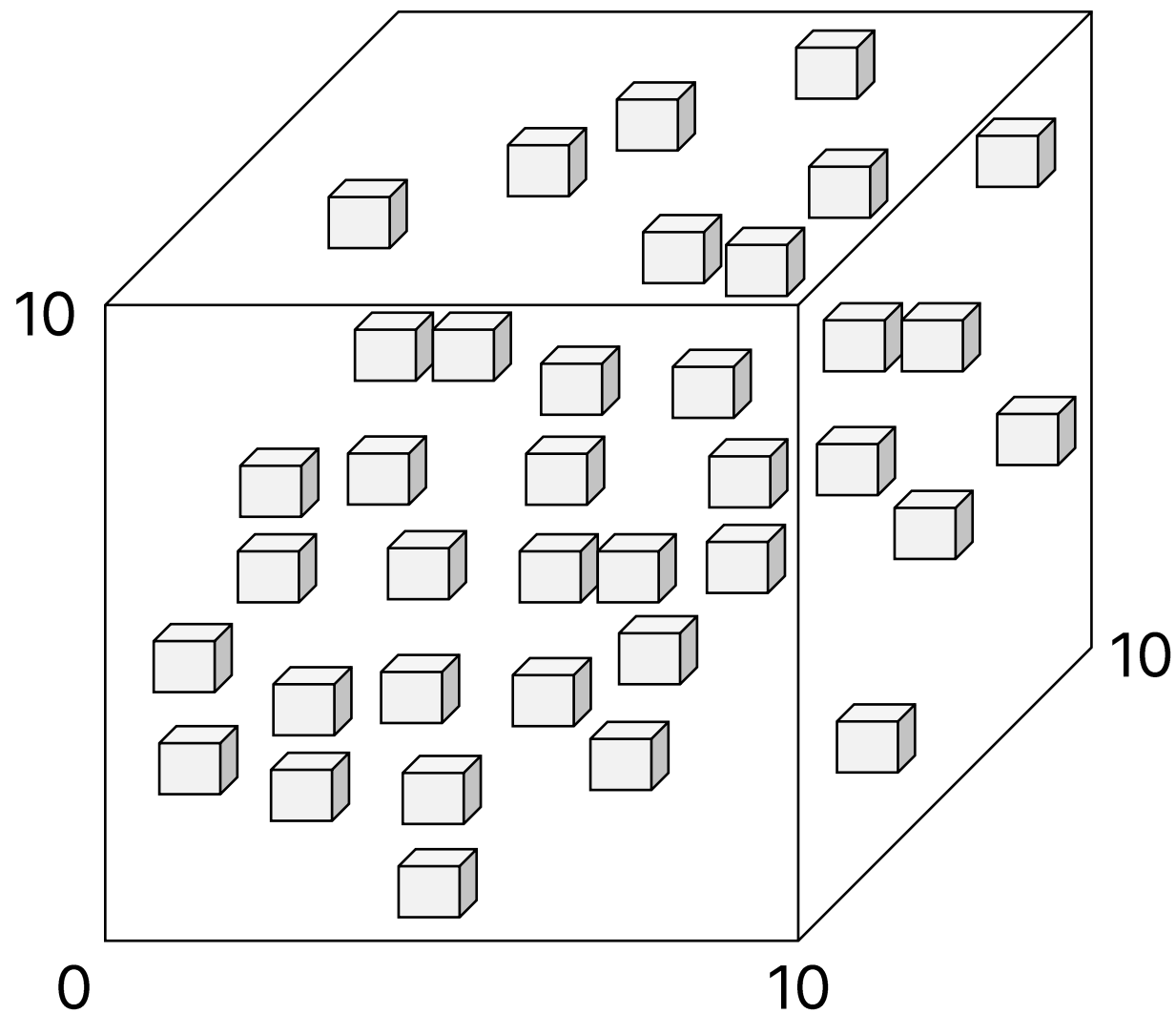
차원 축소(dimension reduction)



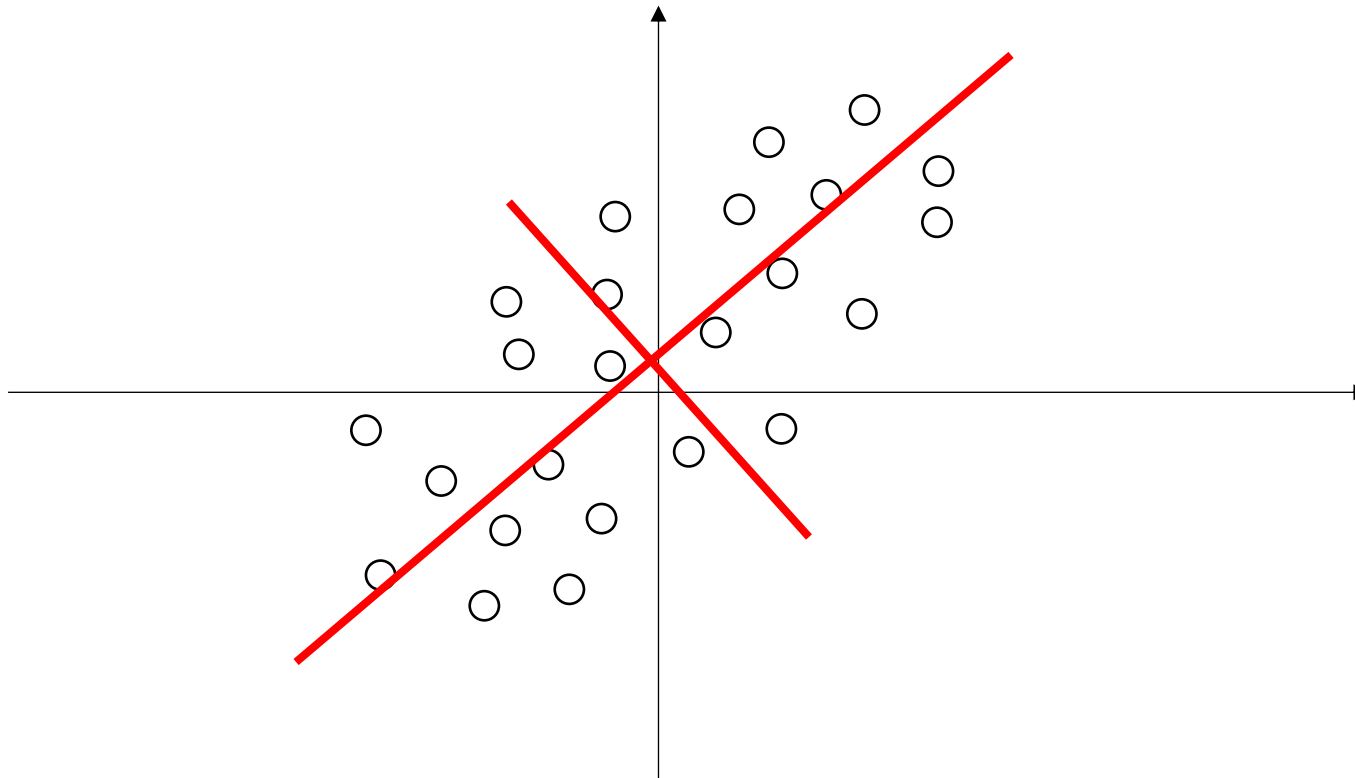
Section
주성분 분석개념



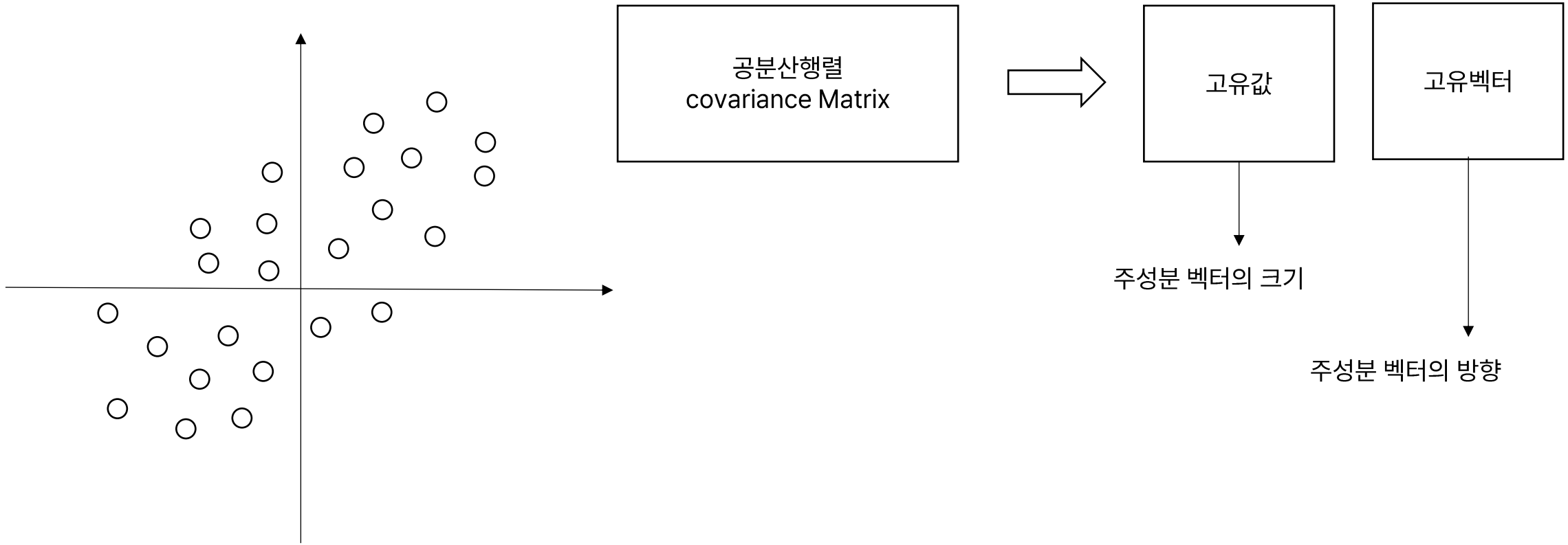
차원 축소(dimension reduction)



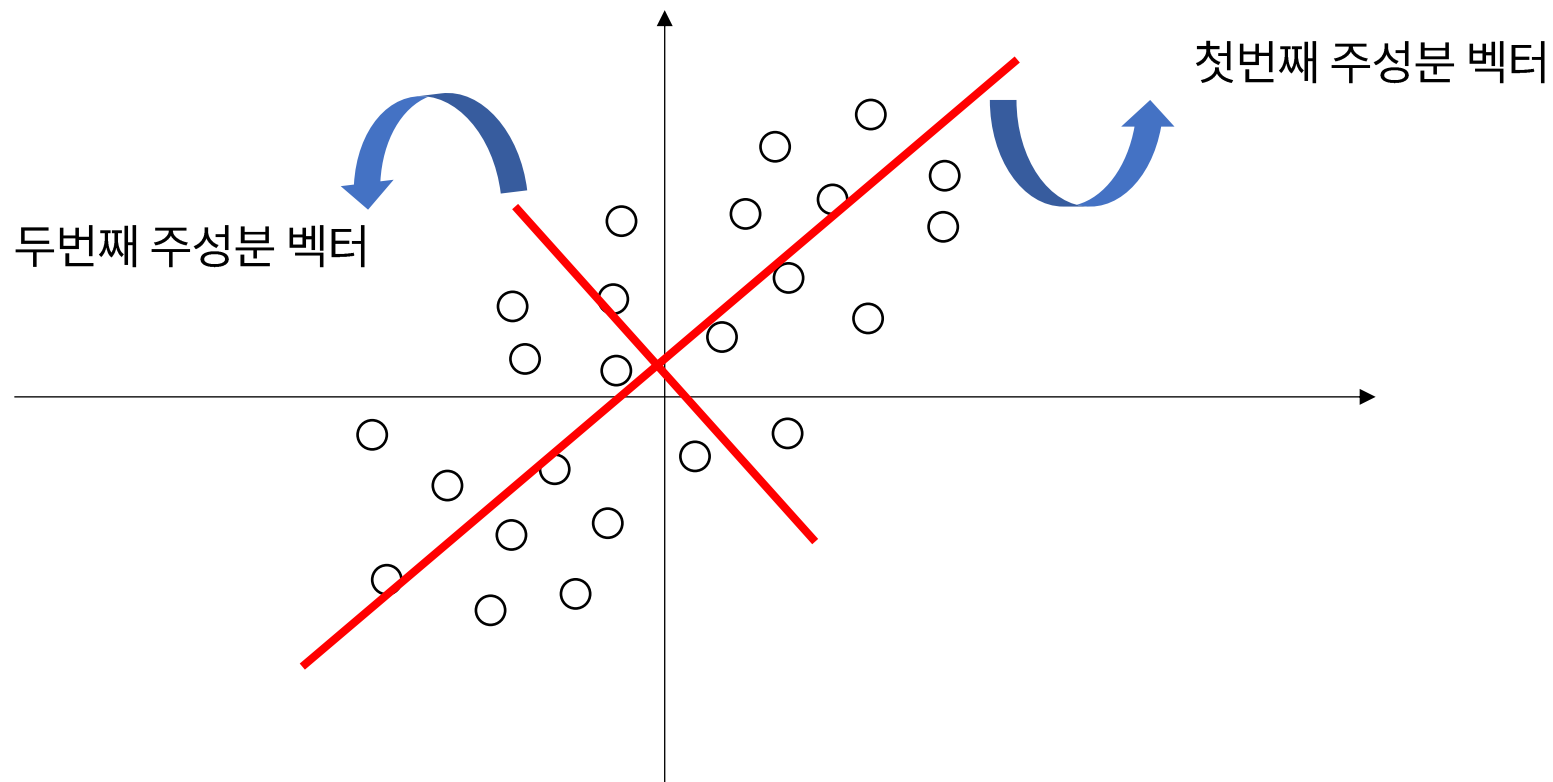
주성분 분석(Principal Component Analysis, PCA)



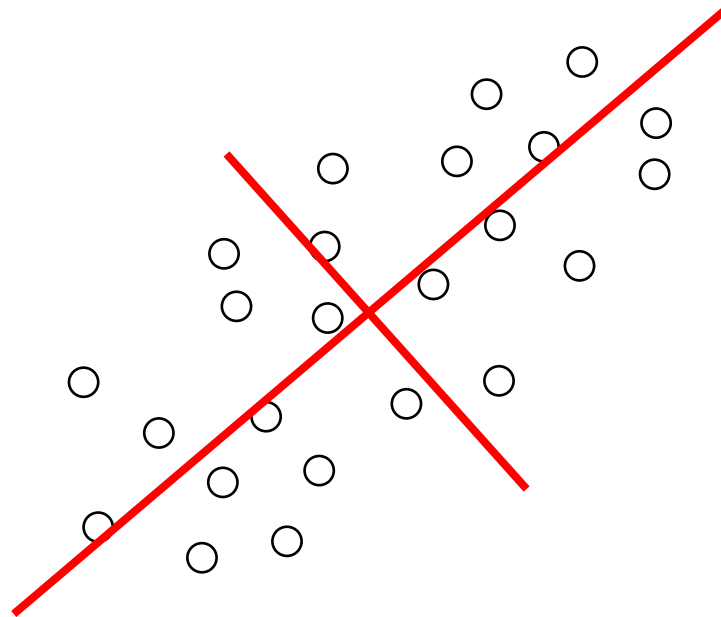
주성분 분석(Principal Component Analysis, PCA)



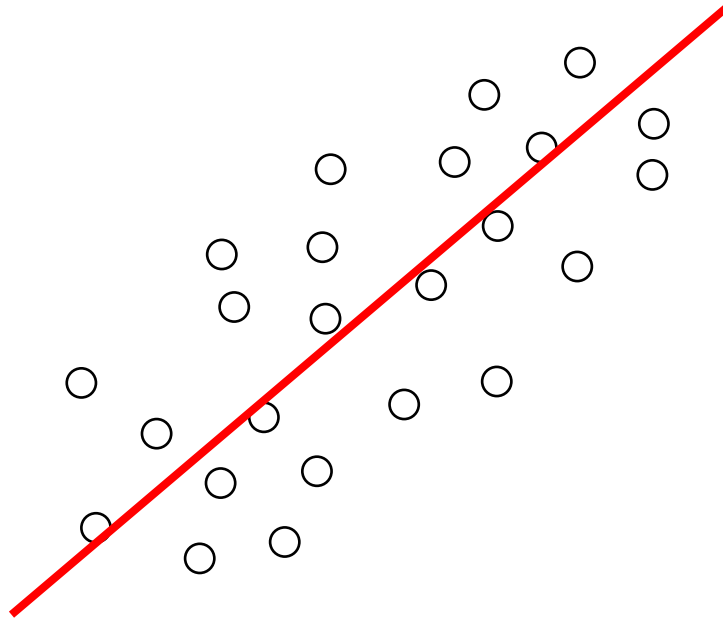
주성분 분석(Principal Component Analysis, PCA)



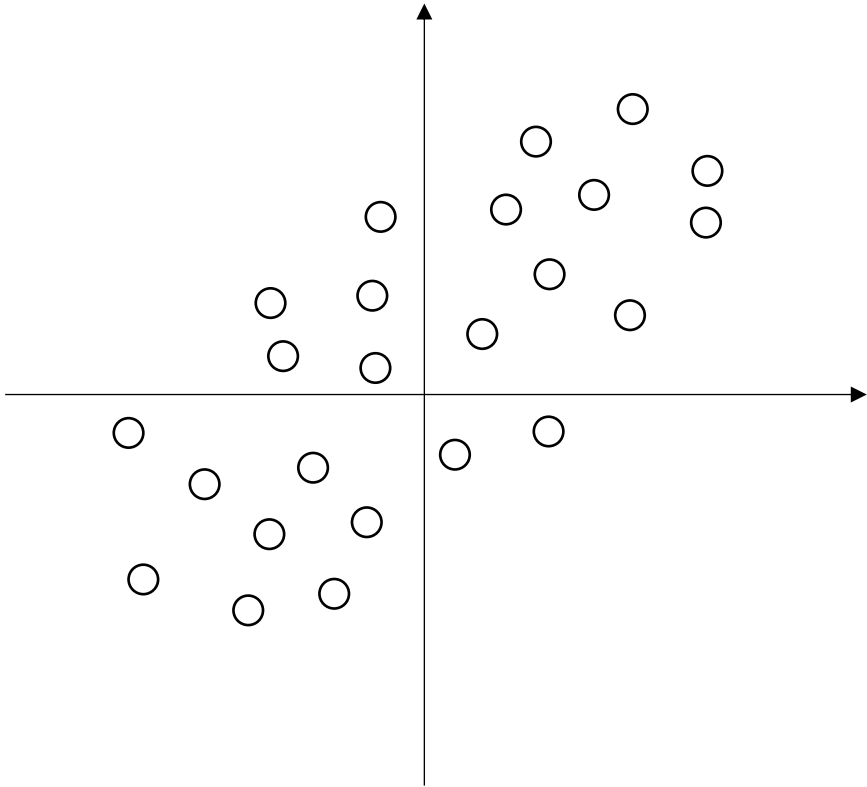
주성분 분석(Principal Component Analysis, PCA)



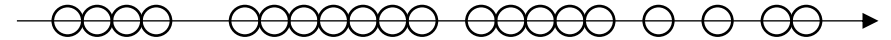
주성분 분석(Principal Component Analysis, PCA)



주성분 분석(Principal Component Analysis, PCA)

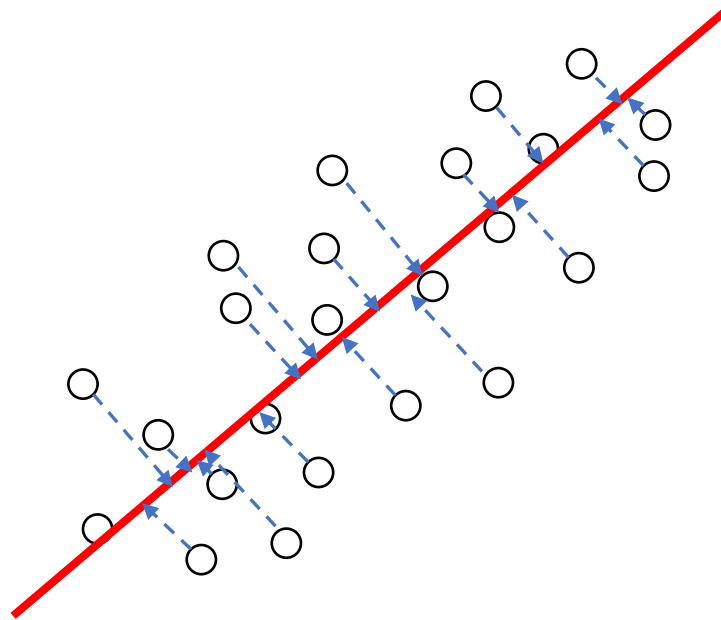


2차원 데이터

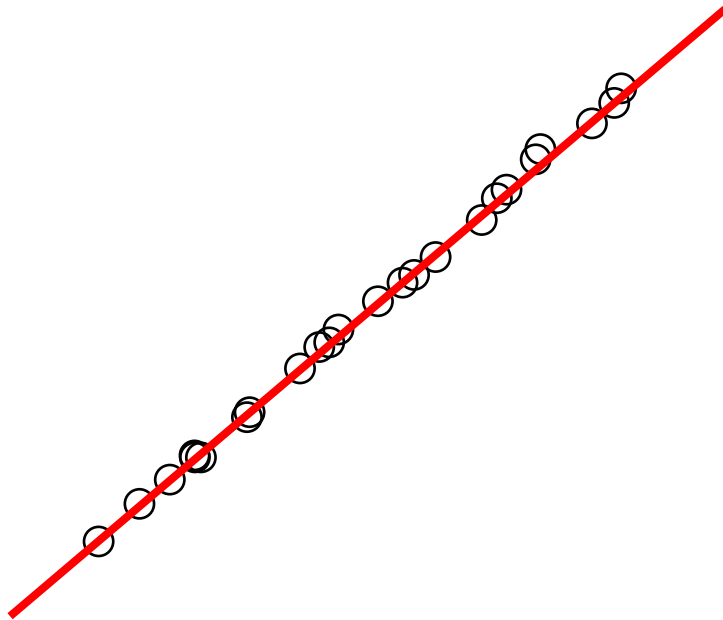


1차원 데이터

주성분 분석(Principal Component Analysis, PCA)

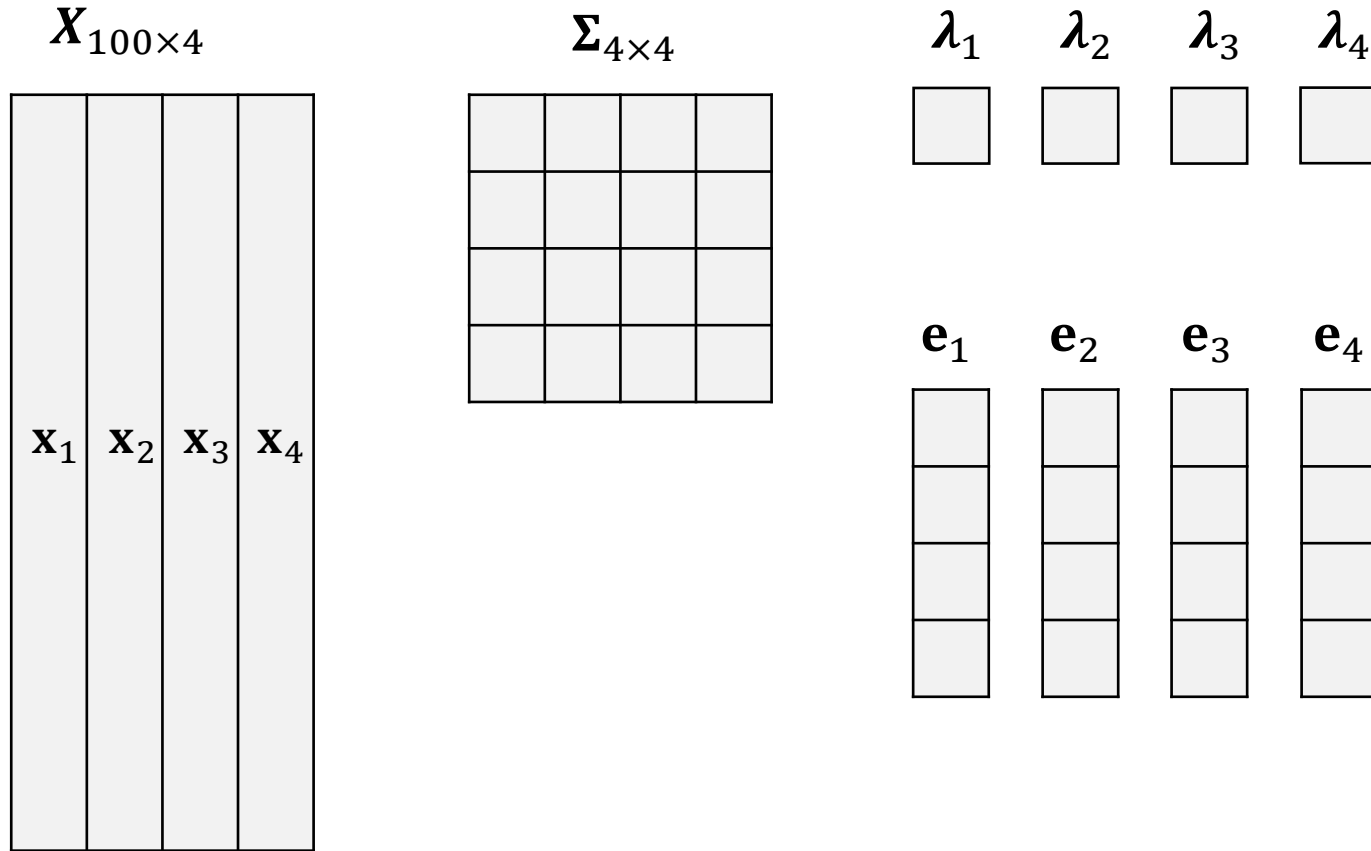


주성분 분석(Principal Component Analysis, PCA)



새로운 좌표축을 기준으로 데이터 표현

주성분 분석(Principal Component Analysis, PCA)



주성분 분석(Principal Component Analysis, PCA)

$$\begin{matrix} \text{y}_1 & = & X & \text{e}_1 \\ 100 \times 1 & & 100 \times 4 & 4 \times 1 \end{matrix}$$

AI/BigData 인텐시브 과정

Section 7. 주성분 분석

Section 7-2. 주성분 분석 실습

주성분 분석(Principal Component Analysis, PCA)

In [1]: ▶

```
1 import pandas as pd
2 df = pd.read_csv("./data/wine_data.csv")
```

In [2]: ▶

```
1 df
```

Out[2]:

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	c
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	

178 rows × 14 columns

주성분 분석(Principal Component Analysis, PCA)

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

주성분 분석(Principal Component Analysis, PCA)

```
In [4]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [5]: ▶ 1 #데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```

주성분 분석(Principal Component Analysis, PCA)

```
In [6]: ▶ 1 # PCA
          2 from sklearn.decomposition import PCA
          3 pca = PCA(n_components=2)
          4 pca.fit(X_tn_std)
          5 X_tn_pca = pca.transform(X_tn_std)
          6 X_te_pca = pca.transform(X_te_std)
```

```
In [7]: ▶ 1 # 차원축소 확인
          2 print(X_tn_std.shape)
          3 print(X_tn_pca.shape)
```

(133, 13)

(133, 2)

주성분 분석(Principal Component Analysis, PCA)

[9]: # 고유벡터 확인

```
print(pca.components_)
```

```
[[ 0.14285263 -0.24443441 -0.01392104 -0.24517361  0.1315099  0.38923734  
 0.42471766 -0.30128613  0.30523465 -0.10462106  0.30267149  0.37266737  
 0.29150867]  
 [ 0.50194071  0.23015832  0.31022311 -0.03872248  0.23858952  0.09688729  
 0.01451474  0.03693926  0.0584635   0.54240798 -0.28168064 -0.16944035  
 0.35182263]]
```

[10]: # 고윳값

```
print(pca.explained_variance_)
```

```
[4.88325506 2.45465553]
```

[11]: # 설명된 분산 비율

```
print(pca.explained_variance_ratio_)
```

```
[0.37281068 0.18739996]
```

주성분 분석(Principal Component Analysis, PCA)

In [12]: ▶

```
1 # 데이터프레임 생성
2 pca_columns = ['pca_comp1', 'pca_comp2']
3 X_tn_pca_df = pd.DataFrame(X_tn_pca, columns=pca_columns)
4 y_tn1 = y_tn.reset_index(drop=True)
5 X_tn_pca_df['target'] = y_tn1
6 X_tn_pca_df.head(10)
```

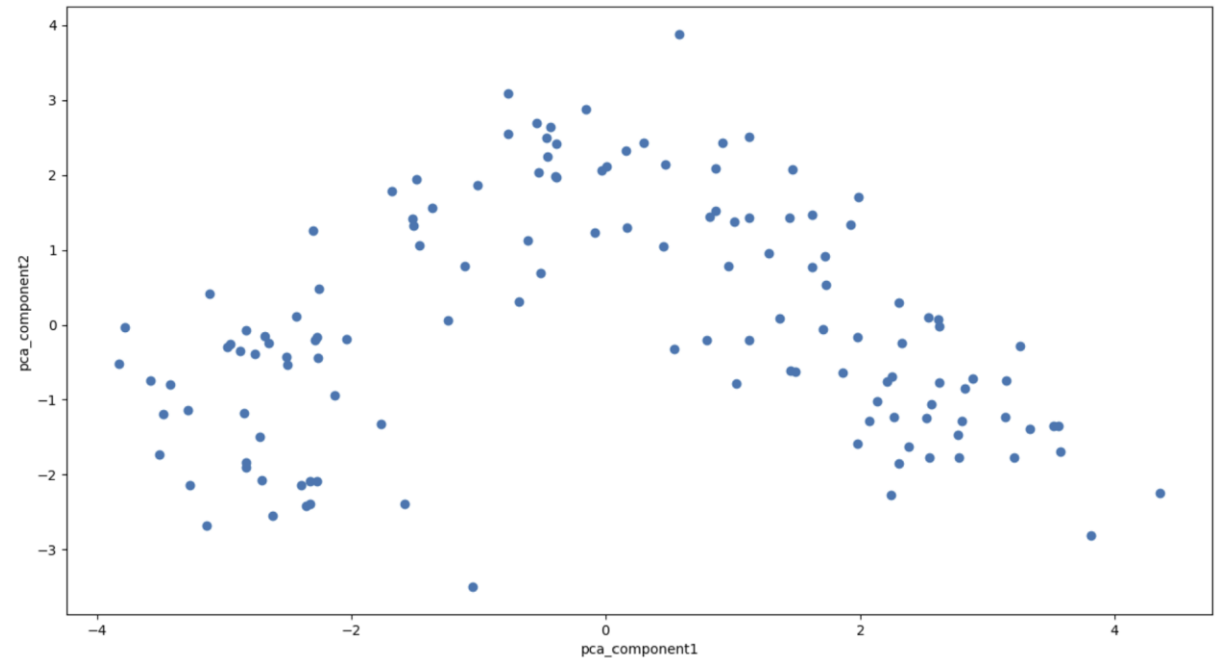
Out[12]:

	pca_comp1	pca_comp2	target
0	2.128500	-1.019732	0
1	-2.948125	-0.254807	2
2	1.919795	1.337012	1
3	-2.133376	-0.945897	2
4	2.236803	-2.269839	0
5	-2.287471	-0.199359	2
6	-2.972450	-0.296523	2
7	2.618098	-0.776070	0
8	-1.047055	-3.490933	2
9	2.609816	0.073457	0

주성분 분석(Principal Component Analysis, PCA)

In [13]: ▶

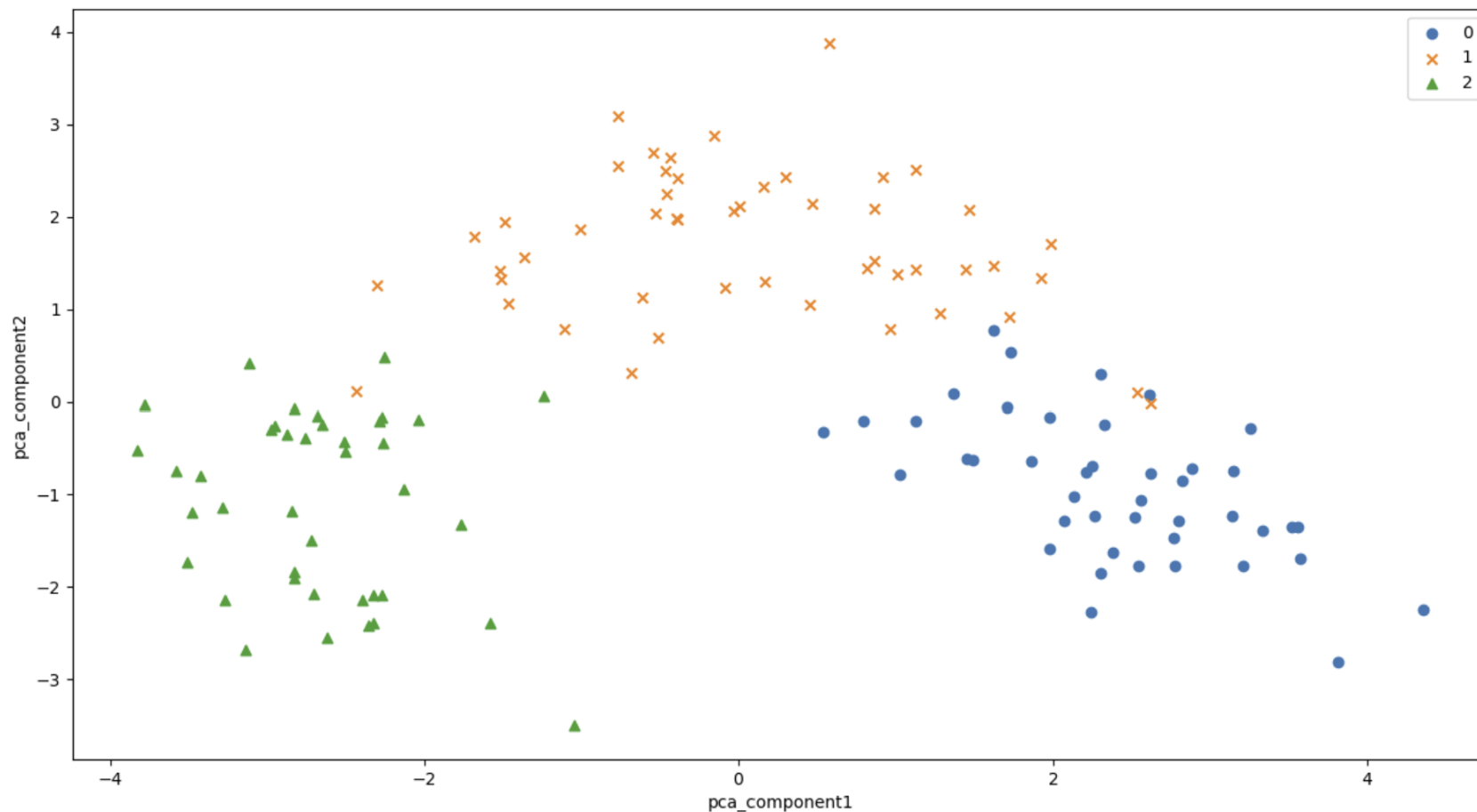
```
1 # 라벨 미적용 PCA 데이터
2 import matplotlib.pyplot as plt
3 plt.figure(figsize=(15,8))
4 plt.scatter(X_tn_pca_df['pca_comp1'],
5             X_tn_pca_df['pca_comp2'],
6             marker='o')
7 plt.xlabel('pca_component1')
8 plt.ylabel('pca_component2')
9 plt.show()
```



Section

주성분분석실습

```
In [14]: ▶ 1 # 라벨 적용 PCA 플랏
2 df = X_tn_pca_df
3 df_0 = df[df['target']==0]
4 df_1 = df[df['target']==1]
5 df_2 = df[df['target']==2]
6
7 X_11 = df_0['pca_comp1']
8 X_12 = df_1['pca_comp1']
9 X_13 = df_2['pca_comp1']
10
11 X_21 = df_0['pca_comp2']
12 X_22 = df_1['pca_comp2']
13
14 target_0 = 0
15 target_1 = 1
16 target_2 = 2
17 X_23 = df_2['pca_comp2']
18
19 plt.figure(figsize=(15,8))
20 plt.scatter(X_11, X_21,
21             marker='o',
22             label=target_0)
23 plt.scatter(X_12, X_22,
24             marker='x',
25             label=target_1)
26 plt.scatter(X_13, X_23,
27             marker='^',
28             label=target_2)
29 plt.xlabel('pca_component1')
30 plt.ylabel('pca_component2')
31 plt.legend()
32 plt.show()
```



PCA + 지도학습

In [15]: ▶

```

1 # 학습
2 from sklearn.neighbors import KNeighborsClassifier
3 clf_knn = KNeighborsClassifier(n_neighbors=2)
4 clf_knn.fit(X_tn_pca, y_tn)

```

Out[15]:

KNeighborsClassifier(n_neighbors=2)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [16]: ▶

```

1 # 예측
2 knn_pred = clf_knn.predict(X_te_pca)
3 print(knn_pred)

```

```

[0 2 1 0 1 0 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0
 1 0 0 0 2
 1 1 2 0 0 1 1 1]

```

In [17]: ▶

```

1 # 정확도
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, knn_pred)
4 print(accuracy)

```

0.9777777777777777

AI/BigData 인텐시브 과정

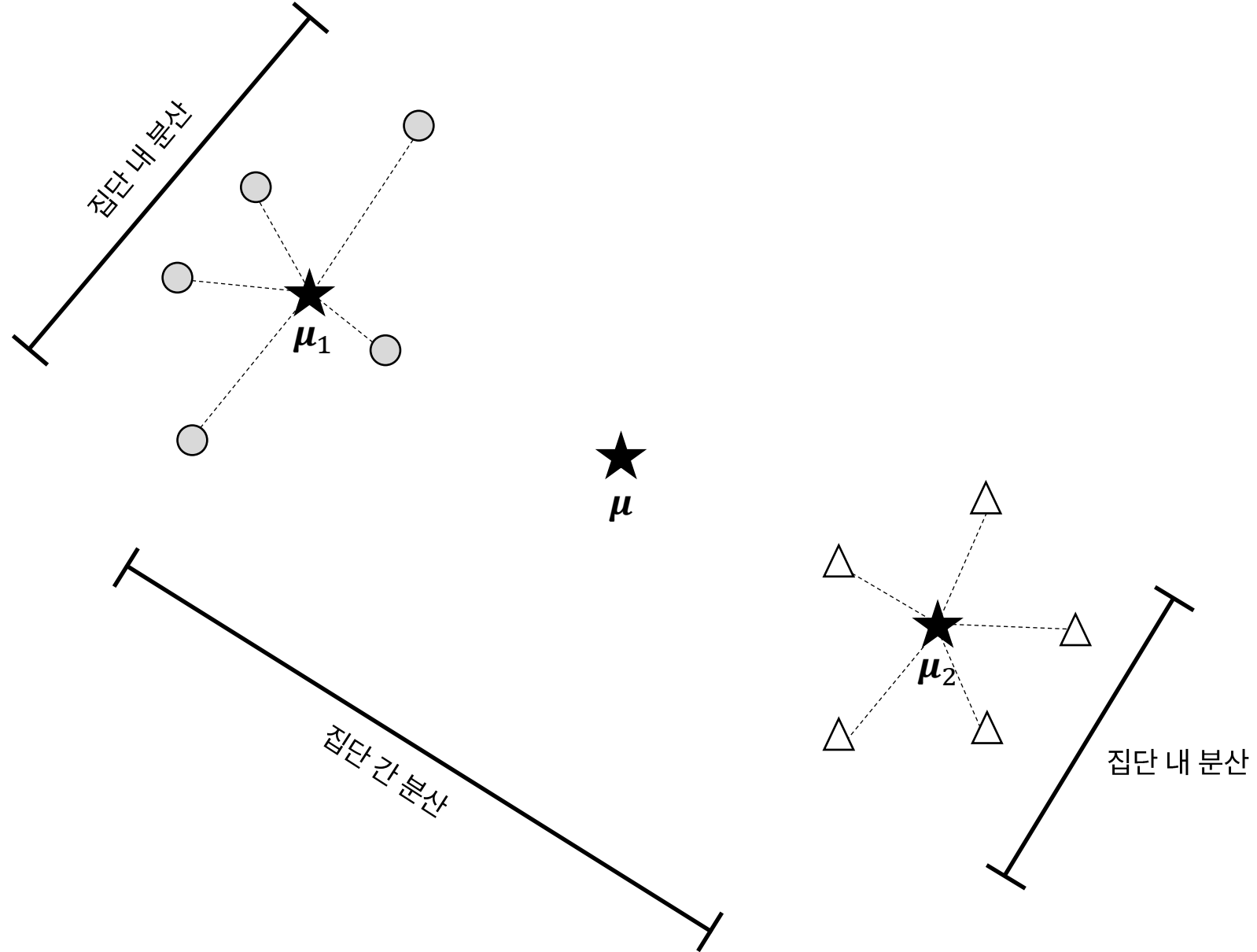
Section 8. LDA

Section 8-1. LDA 개념

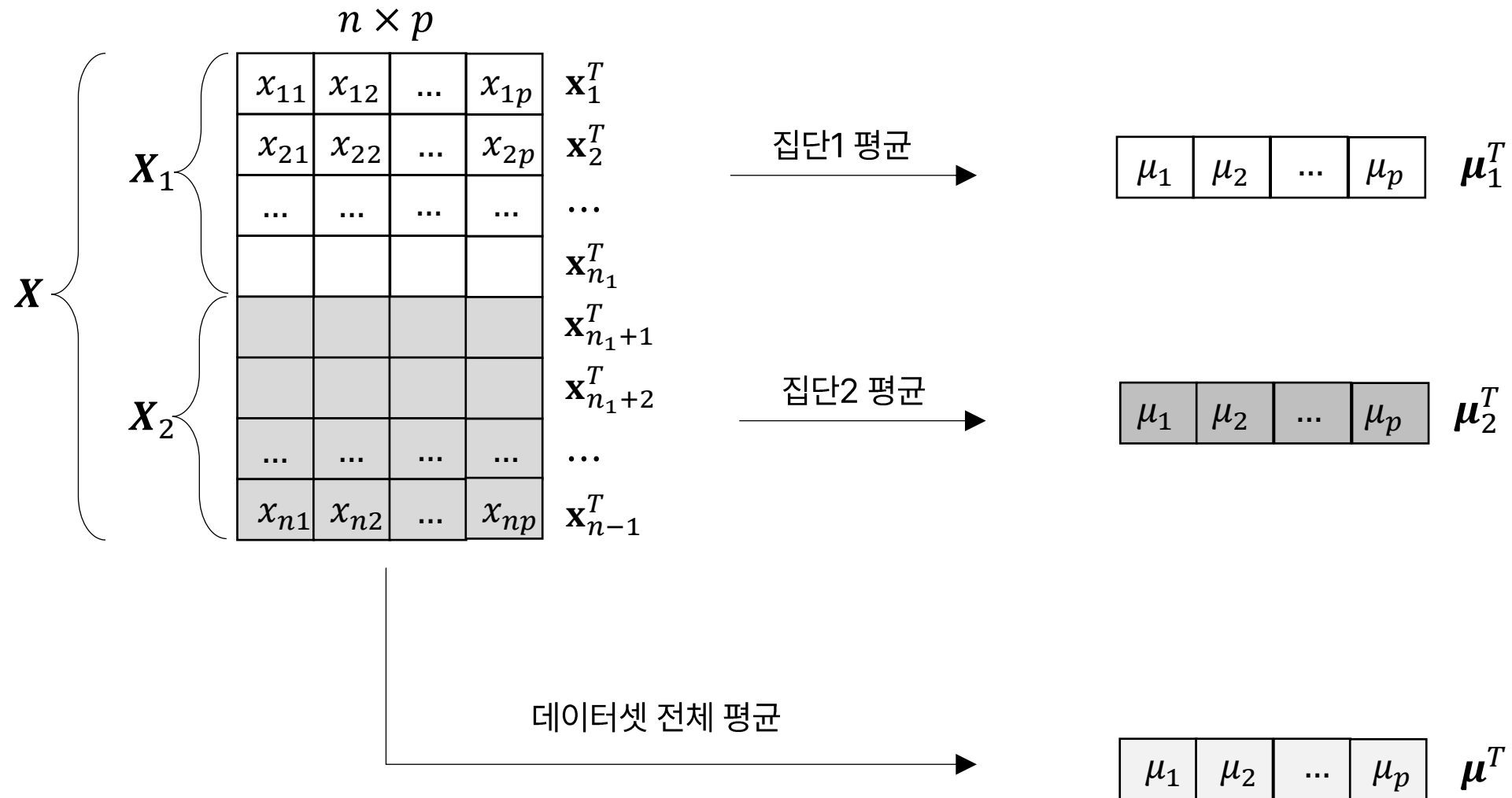
LDA(Linear Discriminant Analysis)

- LDA는 선형 판별 분석이라고 부름
- 선형 판별이라는 말은 데이터 포인트가 속하는 클래스를 구분하는 판별함수가 선형(linear) 형태의 함수라는 뜻
- LDA는 차원 축소 알고리즘을 활용한 지도학습이라고 볼 수 있음
- 데이터 셋의 기존 공간으로부터 집단 간 분산과 집단 내 분산의 비율을 최대화 하는 새로운 공간으로 차원 축소

Section
LDA 개념



LDA(Linear Discriminant Analysis)



집단 간 분산 구하기

i번째 집단의 집단 간 분산이 의미하는 것은 데이터 셋 전체 평균 μ 와 i번째 집단에 속하는 데이터의 평균 μ_i 간의 차이

LDA는 기존 데이터 셋 공간보다 더 작은 차원의 공간 중 집단 간 거리를 최대화하는 공간을 찾는 것이 목적

집단 간 분산 구하기

$$\left\{ n_1 \times \left[\begin{array}{c} \mu_1 \\ \mu_1 \\ \mu_2 \\ \vdots \\ \mu_p \end{array} - \begin{array}{c} \mu \\ \mu_1 \\ \mu_2 \\ \vdots \\ \mu_p \end{array} \right] \times \left[\begin{array}{cccc} \mu_1^T & & & \\ \mu_1 & \mu_2 & \dots & \mu_p \end{array} - \begin{array}{cccc} \mu^T & & & \\ \mu_1 & \mu_2 & \dots & \mu_p \end{array} \right] \right\} \\
 + \left\{ n_2 \times \left[\begin{array}{c} \mu_2 \\ \mu_1 \\ \mu_2 \\ \vdots \\ \mu_p \end{array} - \begin{array}{c} \mu \\ \mu_1 \\ \mu_2 \\ \vdots \\ \mu_p \end{array} \right] \times \left[\begin{array}{cccc} \mu_2^T & & & \\ \mu_1 & \mu_2 & \dots & \mu_p \end{array} - \begin{array}{cccc} \mu^T & & & \\ \mu_1 & \mu_2 & \dots & \mu_p \end{array} \right] \right\} = \Sigma_B$$

$p \times p$

집단 내 분산

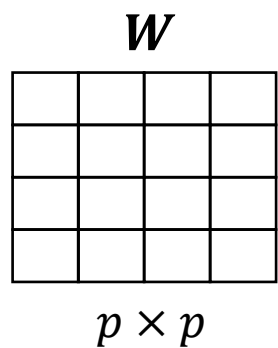
i 번째 집단 내 분산은 각 집단의 평균과 데이터 포인트 간의 차이
LDA는 집단 내 분산을 최소화하는 공간을 찾는 것이 목적

$$\left\{ \left[\begin{array}{c|c} X_1 & \mu_1 \\ \hline \begin{smallmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{smallmatrix} & \begin{smallmatrix} \mu_1 & \mu_2 & \dots & \mu_p \end{smallmatrix} \end{array} \right] \times \left[\begin{array}{c|c} X_1^T & \mu_1^T \\ \hline \begin{smallmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{smallmatrix} & \begin{smallmatrix} \mu_1 \\ \mu_2 \\ \dots \\ \mu_p \end{smallmatrix} \end{array} \right] \right\} \\
 + \left\{ \left[\begin{array}{c|c} X_2 & \mu_2 \\ \hline \begin{smallmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{smallmatrix} & \begin{smallmatrix} \mu_1 & \mu_2 & \dots & \mu_p \end{smallmatrix} \end{array} \right] \times \left[\begin{array}{c|c} X_2^T & \mu_2^T \\ \hline \begin{smallmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{smallmatrix} & \begin{smallmatrix} \mu_1 \\ \mu_2 \\ \dots \\ \mu_p \end{smallmatrix} \end{array} \right] \right\} = \begin{array}{c} \Sigma_W \\ \hline \begin{smallmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{smallmatrix} \\ p \times p \end{array}$$

LDA(Linear Discriminant Analysis)

$$\begin{array}{c} \Sigma_W^{-1} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \\ p \times p \end{array} \times \begin{array}{c} \Sigma_B \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \\ p \times p \end{array} = \begin{array}{c} W \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \\ p \times p \end{array}$$

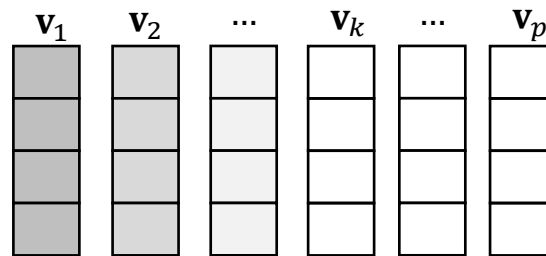
LDA(Linear Discriminant Analysis)



고윳값 분해



λ_1 λ_2 ... λ_k ... λ_p

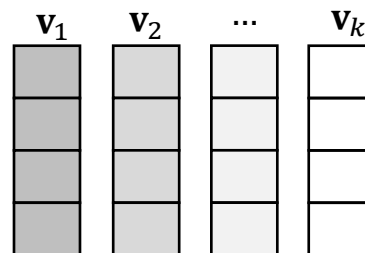
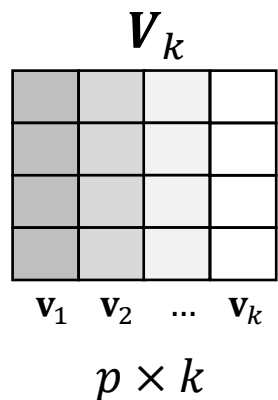
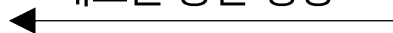


k 개 추출



λ_1 λ_2 ... λ_k

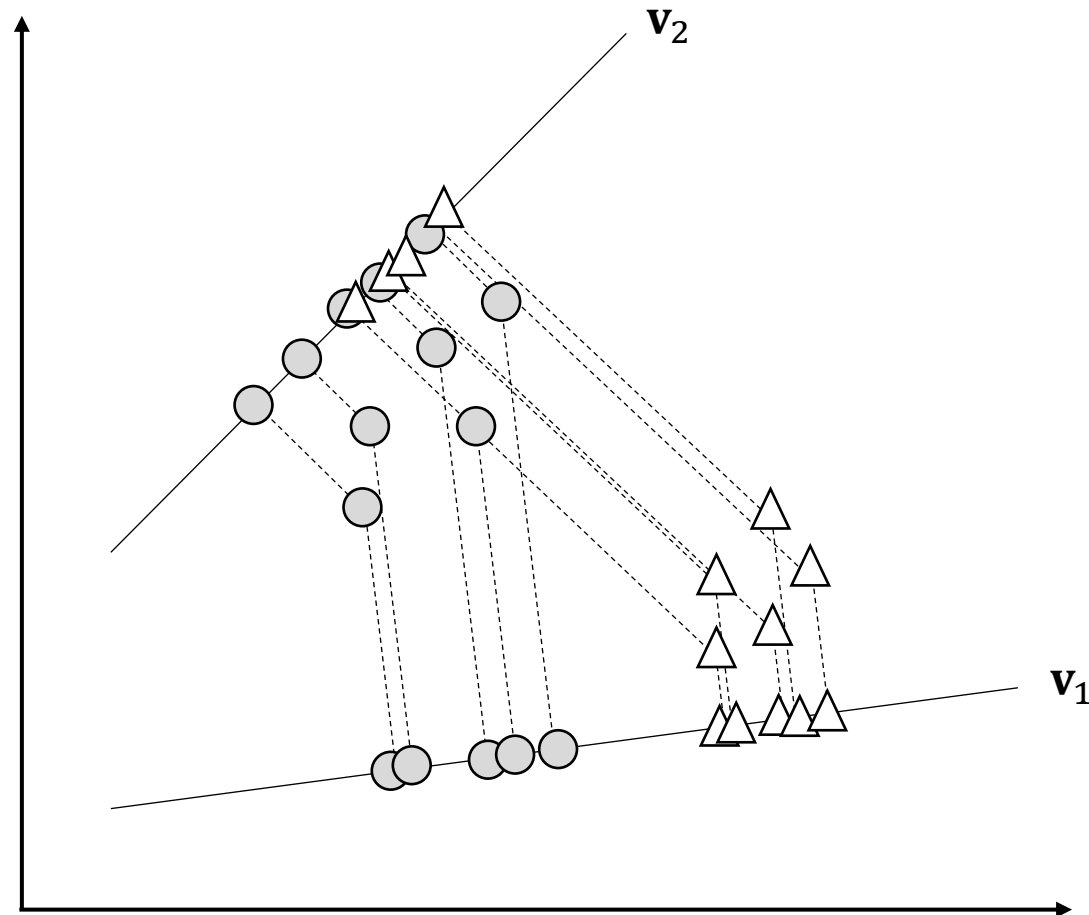
새로운 공간 생성



LDA(Linear Discriminant Analysis)

$$\begin{array}{c} \mathbf{X} \\ \begin{array}{c} \mathbf{X}_1 \\ \mathbf{X}_2 \end{array} \left\{ \begin{array}{c} x_{11} \ x_{12} \ \dots \ x_{1k} \ \dots \ x_{1p} \\ x_{21} \ x_{22} \ \dots \ x_{2k} \ \dots \ x_{2p} \\ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \\ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \\ x_{n1} \ x_{n2} \ \dots \ x_{nk} \ \dots \ x_{np} \end{array} \right. \\ n \times p \end{array} \times \begin{array}{c} \mathbf{V}_k \\ \begin{array}{c} \mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_k \end{array} \\ p \times k \end{array} = \begin{array}{c} \mathbf{X}' \\ \begin{array}{c} \mathbf{X}_1 \\ \mathbf{X}_2 \end{array} \left\{ \begin{array}{c} x_{11} \ x_{12} \ \dots \ x_{1k} \\ x_{21} \ x_{22} \ \dots \ x_{2k} \\ \dots \ \dots \ \dots \ \dots \\ \dots \ \dots \ \dots \ \dots \\ x_{n1} \ x_{n2} \ \dots \ x_{nk} \end{array} \right. \\ n \times k \end{array}$$

LDA(Linear Discriminant Analysis)



AI/BigData 인텐시브 과정

Section 8. LDA

Section 8-2. LDA 실습

LDA(Linear Discriminant Analysis)

```
In [48]: 1 import pandas as pd
          2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [49]: 1 df
```

Out[49]:

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	c
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	

178 rows × 14 columns

LDA(Linear Discriminant Analysis)

```
In [50]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2             'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

```
In [51]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [52]: ▶ 1 # 데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```

LDA(Linear Discriminant Analysis)

```
In [6]: ▶ 1 # LDA 학습
        2 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
        3 lda = LinearDiscriminantAnalysis()
        4 lda.fit(X_tn_std, y_tn)
```

```
Out[6]: ▼ LinearDiscriminantAnalysis
        LinearDiscriminantAnalysis()
```

몇 차원으로 줄일지 설정 안하나요?

```
In [7]: ▶ 1 # 예측
        2 pred_lda = lda.predict(X_te_std)
```

```
In [8]: ▶ 1 # LDA 적용 후 정확도
        2 from sklearn.metrics import accuracy_score
        3 accuracy_lda = accuracy_score(y_te, pred_lda)
        4 print(accuracy_lda)
```

0.9777777777777777

LDA(Linear Discriminant Analysis)

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

```
class sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver='svd', shrinkage=None, priors=None,
n_components=None, store_covariance=False, tol=0.0001, covariance_estimator=None)
```

[\[source\]](#)

n_components : int, default=None

Number of components ($\leq \min(n_classes - 1, n_features)$) for dimensionality reduction. If None, will be set to $\min(n_classes - 1, n_features)$. This parameter only affects the `transform` method.

차원을 따로 설정하지 않을 경우 기본 설정

$$\min(\text{타겟 개수} - 1, \text{피쳐 개수}) = \min(3-1, 13) = \min(2, 13) = 2$$

LDA로 차원축소 + 지도학습

```
In [53]: ▶ 1 # LDA
          2 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
          3 lda = LinearDiscriminantAnalysis()
          4 lda.fit(X_tn_std, y_tn)
          5 X_tn_lda = lda.transform(X_tn_std)
          6 X_te_lda = lda.transform(X_te_std)
```

```
In [54]: ▶ 1 # 차원축소 전후 데이터 차원 비교
          2 print(X_tn_std.shape)
          3 print(X_tn_lda.shape)
```

(133, 13)

(133, 2)

LDA로 차원축소 + 지도학습

In [55]: ▶

```
1 # LDA 상수항  
2 print(lda.intercept_)
```

```
[-10.43672342 -4.06646567 -11.86481563]
```

In [56]: ▶

```
1 # LDA 가중 벡터  
2 print(lda.coef_)
```

```
[[ 2.08067239e+00  9.35053476e-03  1.21559585e+00 -2.93109927e+00  
   9.76489959e-02 -2.70947375e+00  8.33641080e+00  9.59274709e-01  
  -1.31511222e+00 -1.57641572e+00  6.01209963e-02  4.16138211e+00  
   4.94293451e+00]  
 [-1.50898343e+00 -1.03364308e+00 -1.42513305e+00  1.31234341e+00  
  -8.24060552e-03  3.10104765e-01  4.63726906e-01  1.19450044e-01  
   7.45392589e-01 -1.48434856e+00  8.54448096e-01  1.72053276e-01  
  -2.18465736e+00]  
 [-3.50493538e-01  1.28200202e+00  4.74650776e-01  1.51050245e+00  
  -9.46719137e-02  2.52505333e+00 -9.54130024e+00 -1.18053287e+00  
   4.82004904e-01  3.55008261e+00 -1.13269019e+00 -4.68855236e+00  
  -2.58283290e+00]]
```

LDA로 차원축소 + 지도학습

In [60]: ▶

```
1 # LDA 데이터 셋
2 lda_columns = ['lda_comp1', 'lda_comp2']
3 X_tn_lda_df = pd.DataFrame(X_tn_lda, columns=lda_columns)
4 X_tn_lda_df['target'] = y_tn.reset_index(drop=True)
5 X_tn_lda_df.head(5)
```

Out[60]:

	lda_comp1	lda_comp2	target
0	3.757230	1.782242	0
1	-3.717800	0.855757	2
2	0.044172	-2.022490	1
3	-3.502951	1.999101	2
4	3.875662	2.650775	0

LDA로 차원축소 + 지도학습

In [61]: ▶

```
1 from sklearn.ensemble import RandomForestClassifier
2 # 학습
3 clf_rf_lda = RandomForestClassifier(max_depth=2,
4                                     random_state=0)
5 clf_rf_lda.fit(X_tn_lda, y_tn)
6
7 # 예측
8 pred_rf_lda = clf_rf_lda.predict(X_te_lda)
```

In [62]: ▶

```
1 # LDA 적용 후 정확도
2 from sklearn.metrics import accuracy_score
3 accuracy_lda = accuracy_score(y_te, pred_rf_lda)
4 print(accuracy_lda)
```

0.9777777777777777

감사합니다.

Q & A